

V. Dynamic Programming (DP) Patterns

PART A — DYNAMIC PROGRAMMING MASTER NOTES

A1. What Is It?

Dynamic Programming (DP) is an optimization technique that solves complex problems by breaking them into **overlapping subproblems** and storing their results to avoid redundant computation.

Core Idea: - If a problem can be broken into smaller subproblems, and those subproblems repeat, solve each one **once** and **store** the result.

Why it is used: - Converts exponential brute force into polynomial time. - Eliminates redundant recursive calls.

What problem types it solves: - Counting problems (how many ways?) - Optimization problems (minimum cost, maximum profit) - Decision problems (is it possible?) - Constructing solutions (print all ways)

Two properties a problem must have for DP: 1. **Optimal Substructure** — Optimal solution can be built from optimal solutions of subproblems. 2. **Overlapping Subproblems** — Same subproblems are solved multiple times.

Three approaches (in order of development):

```
Recursion (plain)
  ↓ Add memo table
Memoization (Top-Down DP)
  ↓ Convert to iterative
Tabulation (Bottom-Up DP)
  ↓ Reduce space
Space Optimized
```

A2. Core Operations / Concepts

Recursion (Brute Force)

```
def solve(state):
    if base_case:
        return base_value
    return combine(solve(smaller_state_1), solve(smaller_state_2))
```

- Pure recursion without storing results.
- Often exponential time: $O(2^n)$ or $O(n!)$.

Memoization (Top-Down)

```
def solve(state, memo={}):
    if state in memo:
        return memo[state]
    if base_case:
        return base_value
    memo[state] = combine(solve(smaller_state_1), solve(smaller_state_2))
    return memo[state]
```

- Same recursive structure, add a dictionary/array to cache results.
- Time reduces to $O(\text{number of unique states} \times \text{work per state})$.

Tabulation (Bottom-Up)

```
dp = [base_values]
for state in order:
    dp[state] = combine(dp[smaller_state_1], dp[smaller_state_2])
return dp[final_state]
```

- Iterative, fills table from smallest subproblem to largest.
- No recursion stack overhead.

Space Optimization

When `dp[i]` depends only on `dp[i-1]` (or a few previous states), reduce from $O(n)$ to $O(1)$ space.

```
prev, curr = base_values
for i in range(2, n+1):
    curr = combine(prev_values)
    prev = curr
```

Python-Specific Tools

Tool	Usage	When to Use
<code>@lru_cache(None)</code>	Auto-memoization decorator	Top-down DP on functions
<code>dict</code>	Manual memo table	When states are complex tuples
<code>list</code>	DP table	Bottom-up tabulation
<code>float('inf')</code>	Initialize for min problems	Coin change, path sum
<code>float('-inf')</code>	Initialize for max problems	Maximum subarray

```
from functools import lru_cache

@lru_cache(None)
def solve(i):
    # recursive calls automatically cached
    pass
```

A3. Time Complexity

Approach	Time	Space
----------	------	-------

Plain Recursion	$O(2^n)$ or worse	$O(n)$ stack
Memoization	$O(\text{states} \times \text{work})$	$O(\text{states}) + O(n)$ stack
Tabulation	$O(\text{states} \times \text{work})$	$O(\text{states})$
Space Optimized	$O(\text{states} \times \text{work})$	$O(1)$ or $O(n)$

General DP complexity formula:

Time = (Number of unique states) $\times$ (Work done per state)
Space = (Table size) + (Recursion stack if top-down)

Common DP complexities by pattern:

Pattern	States	Work/State	Total Time	Space
Fibonacci Style	$O(n)$	$O(1)$	$O(n)$	$O(1)$ optimized
Kadane's	$O(n)$	$O(1)$	$O(n)$	$O(1)$
Coin Change	$O(n \times \text{amount})$ or $O(\text{amount})$	$O(1)$ per coin	$O(n \times \text{amount})$	$O(\text{amount})$
0/1 Knapsack	$O(n \times W)$	$O(1)$	$O(n \times W)$	$O(W)$ optimized
LCS	$O(m \times n)$	$O(1)$	$O(m \times n)$	$O(n)$ optimized
Grid DP	$O(m \times n)$	$O(1)$	$O(m \times n)$	$O(n)$ optimized

Interval DP	$O(n^2)$ or $O(n^3)$	$O(n)$	$O(n^3)$	$O(n^2)$
LIS	$O(n^2)$ or $O(n \log n)$	$O(1)$ or $O(\log n)$	$O(n \log n)$	$O(n)$

A4. Important Patterns / Variations

Pattern 1: 1D Fibonacci Style

- When: $dp[i]$ depends on $dp[i-1]$, $dp[i-2]$ (or few previous)
- Core idea: Linear recurrence relation
- Complexity: $O(n)$ time, $O(1)$ space optimized
- Example: Climbing Stairs → $dp[i] = dp[i-1] + dp[i-2]$

Pattern 2: Kadane's Algorithm

- When: Maximum/minimum subarray sum
- Core idea: Extend current subarray or start new
- Complexity: $O(n)$ time, $O(1)$ space
- Example: $max_ending_here = \max(nums[i], max_ending_here + nums[i])$

Pattern 3: Unbounded Knapsack / Coin Change

- When: Unlimited supply of items, reach target sum/value
- Core idea: For each item, include it (stay) or skip it (move)
- Complexity: $O(n \times target)$ time
- Example: Coin Change → $dp[amount] = \min(dp[amount], dp[amount - coin] + 1)$

Pattern 4: 0/1 Knapsack / Subset Sum

- When: Each item used at most once, subset selection
- Core idea: Include or exclude each item
- Complexity: $O(n \times target)$ time

→ Example: Partition Equal Subset Sum → can we pick subset summing to tot

Pattern 5: Word Break

- When: String segmentation using dictionary
- Core idea: Try every prefix, if valid word, recurse on suffix
- Complexity: $O(n^2 \times m)$ where $m = \text{max word length}$
- Example: $dp[i] = \text{any}(dp[j] \text{ and } s[j:i] \text{ in wordDict})$

Pattern 6: LCS (Longest Common Subsequence)

- When: Two sequences, find common/matching elements preserving order
- Core idea: Match characters or skip one
- Complexity: $O(m \times n)$ time
- Example: If match → $1 + dp[i-1][j-1]$, else $\max(dp[i-1][j], dp[i][j-1])$

Pattern 7: Edit Distance

- When: Transform one string to another with insert/delete/replace
- Core idea: Three operations at each mismatch
- Complexity: $O(m \times n)$ time
- Example: $dp[i][j] = \min(\text{insert, delete, replace}) + 1$

Pattern 8: Grid DP

- When: Move through 2D grid (right/down), count paths or minimize cost
- Core idea: $dp[i][j]$ depends on neighbors (above, left)
- Complexity: $O(m \times n)$ time
- Example: $dp[i][j] = dp[i-1][j] + dp[i][j-1]$

Pattern 9: Interval DP

- When: Optimal way to merge/split a range
- Core idea: Try every split point in range $[i, j]$
- Complexity: $O(n^3)$ time
- Example: Burst Balloons → try every last balloon to burst in range

Pattern 10: Catalan Numbers

- When: Count structurally unique trees/expressions/parentheses
- Core idea: Split into left/right with each root
- Complexity: $O(n^2)$ time
- Example: $C(n) = \sum C(i) \times C(n-1-i)$ for i in $[0, n-1]$

Pattern 11: LIS (Longest Increasing Subsequence)

- When: Longest subsequence with ordering constraint
- Core idea: For each element, find best previous element to extend
- Complexity: $O(n \log n)$ with patience sorting / binary search
- Example: Maintain sorted tails array, binary search for position

A5. Recognition Guide

How to Recognize DP

Look for:

- "How many ways..." → Counting DP
- "Minimum/maximum cost/profit/value..." → Optimization DP
- "Is it possible..." → Decision DP (boolean DP)
- "Print all ways..." → DP + backtracking
- Choices at each step (include/exclude, take/skip)
- Problem has overlapping subproblems (same inputs computed repeatedly)
- Greedy doesn't work (local optimal $\neq$ global optimal)

How to Decide Which DP Pattern

Clue in Problem	Pattern
<code>dp[i]</code> depends on <code>dp[i-1]</code> , <code>dp[i-2]</code>	Fibonacci Style
Maximum/minimum contiguous subarray	Kadane's
Unlimited items, reach target	Unbounded Knapsack
Each item used once, subset, target sum	0/1 Knapsack
String segmentation with dictionary	Word Break
Two strings, common/transform	LCS / Edit Distance
Grid traversal, paths, min cost	Grid DP
Merge/split ranges optimally	Interval DP
Count structurally unique structures	Catalan
Longest subsequence with ordering	LIS

How to Choose Recursion vs Memoization vs Tabulation

Step 1: Write the recursive solution first (understand the recurrence)
Step 2: Add memoization (easiest optimization)
Step 3: Convert to tabulation (if stack overflow or need space optimization)
Step 4: Optimize space (if `dp[i]` depends on only few previous states)

A6. Common Interview Traps

1. **Wrong base case** — Forgetting `dp[0] = 1` vs `dp[0] = 0` (counting vs optimization).
2. **Wrong iteration order** — Bottom-up must fill dependencies before current state.
3. **Off-by-one errors** — Array size `n+1` vs `n`, range `(0, n)` vs `(0, n+1)`.
4. **0/1 vs unbounded confusion** — 0/1 knapsack: iterate right-to-left (or 2D).
Unbounded: left-to-right.
5. **Missing `float('inf')` initialization** — For minimization problems, initialize with infinity, not 0.
6. **Forgetting to handle impossible cases** — Return -1 when `dp[target]` is still infinity.
7. **Incorrect state definition** — State must capture all information needed to make decisions.
8. **Not recognizing overlapping subproblems** — If subproblems don't overlap, DP is unnecessary (use divide & conquer).
9. **Confusing permutations vs combinations** — Coin Change II (combinations): outer loop items, inner loop amount. Combination Sum IV (permutations): outer loop amount, inner loop items.
10. **Stack overflow in memoization** — Python default recursion limit is 1000. Use `sys.setrecursionlimit()` or convert to tabulation.
11. **2D DP space optimization direction** — When optimizing 2D to 1D, must iterate in correct direction to avoid using already-updated values.

A7. Pattern Cheat Sheet

Pattern	Recognition	Recurrence Core	Typical Complexity
Fibonacci Style	<code>dp[i] = f(few previous)</code>	<code>dp[i] = dp[i-1] + dp[i-2]</code>	O(n) time, O(1) space

Kadane's	Max/min subarray	$\max(\text{nums}[i], \text{dp}[i-1] + \text{nums}[i])$	$O(n)$ time, $O(1)$ space
Unbounded Knapsack	Unlimited items, target	$\text{dp}[a] = \min/\max(\text{dp}[a], \text{dp}[a-\text{item}] + \text{cost})$	$O(n \times \text{target})$
0/1 Knapsack	Each item once, subset	$\text{dp}[j] = \text{dp}[j]$ or $\text{dp}[j-\text{num}]$	$O(n \times \text{target})$
Word Break	String + dictionary	$\text{dp}[i] = \text{any}(\text{dp}[j] \text{ and } s[j:i] \text{ in dict})$	$O(n^2 \times m)$
LCS	Two sequences match	$1 + \text{dp}[i-1][j-1]$ or $\max(\text{skip})$	$O(m \times n)$
Edit Distance	Transform string	$\min(\text{insert}, \text{delete}, \text{replace})$	$O(m \times n)$
Grid DP	2D grid paths/cost	$\text{dp}[i][j] = f(\text{up}, \text{left})$	$O(m \times n)$
Interval DP	Range merge/split	$\min/\max \text{ over split } k \text{ in } [i, j]$	$O(n^3)$
Catalan	Unique structures count	$\sum C(i) \times C(n-1-i)$	$O(n^2)$

LIS	Longest ordered subseq	Binary search on tails	$O(n \log n)$
-----	------------------------	------------------------	---------------

A8. DP Formula Summary Sheet

All key recurrences in one place for quick revision:

FIBONACCI STYLE

Fibonacci: $dp[i] = dp[i-1] + dp[i-2]$
 Climbing Stairs: $dp[i] = dp[i-1] + dp[i-2]$
 Min Cost Stairs: $dp[i] = cost[i] + \min(dp[i-1], dp[i-2])$
 House Robber: $dp[i] = \max(dp[i-1], dp[i-2] + nums[i])$
 Decode Ways: $dp[i] = (dp[i-1] \text{ if valid1}) + (dp[i-2] \text{ if valid2})$
 Delete and Earn: $dp[i] = \max(dp[i-1], dp[i-2] + points[i])$

KADANE'S

Max Subarray: $dp[i] = \max(nums[i], dp[i-1] + nums[i])$
 Answer: $\max(dp[0..n-1])$

COIN CHANGE / UNBOUNDED KNAPSACK

Coin Change: $dp[a] = \min(dp[a], dp[a - coin] + 1)$ for each coin
 Coin Change II: $dp[a] += dp[a - coin]$ (outer: coins, inner: amount)
 Combination Sum IV: $dp[a] += dp[a - num]$ (outer: amount, inner: nums)

0/1 KNAPSACK / SUBSET SUM

Partition Subset: $dp[j] = dp[j] \text{ or } dp[j - num]$ (iterate j right→left)
 Target Sum: $dp[j] += dp[j - num]$ (count ways, iterate right→left)

WORD BREAK

Word Break: $dp[i] = \text{any}(dp[j] \text{ and } s[j:i] \text{ in wordDict for } j < i)$

LCS

LCS: $\text{if match: } dp[i][j] = 1 + dp[i-1][j-1]$
 $\text{else: } dp[i][j] = \max(dp[i-1][j], dp[i][j-1])$

EDIT DISTANCE

Edit Distance: $\text{if match: } dp[i][j] = dp[i-1][j-1]$
 $\text{else: } dp[i][j] = 1 + \min(dp[i-1][j], \quad \# \text{ delete}$
 $\quad \quad \quad dp[i][j-1], \quad \# \text{ insert}$
 $\quad \quad \quad dp[i-1][j-1]) \quad \# \text{ replace}$

GRID DP

Unique Paths: $dp[i][j] = dp[i-1][j] + dp[i][j-1]$
Min Path Sum: $dp[i][j] = \text{grid}[i][j] + \min(dp[i-1][j], dp[i][j-1])$
Maximal Square: $dp[i][j] = \min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1]) + 1$

INTERVAL DP

Burst Balloons: $dp[i][j] = \max(dp[i][k-1] + dp[k+1][j] +$
 $\quad \quad \quad \text{nums}[i-1] * \text{nums}[k] * \text{nums}[j+1]) \text{ for } k \text{ in } [i, j]$

CATALAN

Unique BSTs: $dp[n] = \sum dp[i] * dp[n-1-i] \text{ for } i \text{ in } [0, n-1]$
Catalan Number: $C(n) = C(2n, n) / (n+1)$

```
LIS ( $O(n^2)$ ):      dp[i] = max(dp[j] + 1) for j < i where nums[j] < nums[i]
LIS ( $O(n \log n)$ ): maintain tails[], bisect_left to find position
```

PART B — PROBLEM SOLUTIONS

Pattern 27: DP - 1D Array (Fibonacci Style)

LC 509. Fibonacci Number

Difficulty: Easy **Pattern:** 1D Fibonacci Style

Problem

Given n , return $F(n)$ where $F(0) = 0$, $F(1) = 1$, and $F(n) = F(n-1) + F(n-2)$.

Approaches

This is the foundational DP problem. Every approach matters for understanding DP itself.

Approach 1 — Recursion (Brute Force)

- Directly apply the definition recursively.
- **Time:** $O(2^n)$ — exponential due to repeated subproblems.
- **Space:** $O(n)$ — recursion stack.

Approach 2 — Memoization (Top-Down)

- Cache computed Fibonacci values.
- **Time:** $O(n)$

- **Space:** $O(n)$

Approach 3 — Tabulation (Bottom-Up)

- Fill array from 0 to n iteratively.
- **Time:** $O(n)$
- **Space:** $O(n)$

Approach 4 — Space Optimized

- Only keep two previous values.
- **Time:** $O(n)$
- **Space:** $O(1)$

Recommended Approach: Space Optimized for interviews, but know all four.

Python Code

Recursion:

```
def fib(n):  
    if n <= 1:  
        return n  
    return fib(n - 1) + fib(n - 2)
```

Memoization:

```
def fib(n, memo={}):  
    if n in memo:  
        return memo[n]  
    if n <= 1:  
        return n  
    memo[n] = fib(n - 1, memo) + fib(n - 2, memo)  
    return memo[n]
```

Tabulation:

```
def fib(n):
    if n <= 1:
        return n
    dp = [0] * (n + 1)
    dp[1] = 1
    for i in range(2, n + 1):
        dp[i] = dp[i - 1] + dp[i - 2]
    return dp[n]
```

Space Optimized:

```
def fib(n):
    if n <= 1:
        return n
    prev2, prev1 = 0, 1
    for _ in range(2, n + 1):
        curr = prev1 + prev2
        prev2 = prev1
        prev1 = curr
    return prev1
```

Complexity

- **Time:** $O(n)$
- **Space:** $O(1)$ (space optimized)

LC 70. Climbing Stairs

Difficulty: Easy **Pattern:** 1D Fibonacci Style

Problem

You can climb 1 or 2 steps at a time. Given n stairs, return the number of **distinct ways** to reach the top.

Approaches

Classic DP problem. Identical structure to Fibonacci.

Approach 1 — Recursion

- At each step, either take 1 or 2 steps.
- $\text{ways}(n) = \text{ways}(n-1) + \text{ways}(n-2)$
- **Time:** $O(2^n)$ — **Space:** $O(n)$

Approach 2 — Memoization

- Cache results of $\text{ways}(n)$.
- **Time:** $O(n)$ — **Space:** $O(n)$

Approach 3 — Tabulation

- **Time:** $O(n)$ — **Space:** $O(n)$

Approach 4 — Space Optimized

- **Time:** $O(n)$ — **Space:** $O(1)$

Recommended Approach: Space Optimized.

Key Observation

$\text{dp}[i] = \text{dp}[i-1] + \text{dp}[i-2]$ with $\text{dp}[1] = 1$, $\text{dp}[2] = 2$. This is exactly Fibonacci shifted by one.

Python Code

Recursion:

```
def climbStairs(n):
    if n <= 2:
        return n
    return climbStairs(n - 1) + climbStairs(n - 2)
```

Memoization:

```
def climbStairs(n, memo={}):
    if n in memo:
        return memo[n]
    if n <= 2:
        return n
    memo[n] = climbStairs(n - 1, memo) + climbStairs(n - 2, memo)
    return memo[n]
```

Tabulation:

```
def climbStairs(n):
    if n <= 2:
        return n
    dp = [0] * (n + 1)
    dp[1], dp[2] = 1, 2
    for i in range(3, n + 1):
        dp[i] = dp[i - 1] + dp[i - 2]
    return dp[n]
```

Space Optimized:

```
def climbStairs(n):
    if n <= 2:
        return n
    prev2, prev1 = 1, 2
    for _ in range(3, n + 1):
        curr = prev1 + prev2
        prev2 = prev1
        prev1 = curr
    return prev1
```

Complexity

- **Time:** $O(n)$
 - **Space:** $O(1)$
-

LC 746. Min Cost Climbing Stairs

Difficulty: Easy **Pattern:** 1D Fibonacci Style

Problem

Given `cost[i]` for standing on step `i`, you can climb 1 or 2 steps. Start from step 0 or step 1. Find the **minimum cost** to reach the top (past the last step).

Approaches

Approach 1 — Recursion

- `minCost(i) = cost[i] + min(minCost(i-1), minCost(i-2))`
- **Time:** $O(2^n)$ — **Space:** $O(n)$

Approach 2 — Memoization

- **Time:** $O(n)$ — **Space:** $O(n)$

Approach 3 — Tabulation

- **Time:** $O(n)$ — **Space:** $O(n)$

Approach 4 — Space Optimized

- **Time:** $O(n)$ — **Space:** $O(1)$

Recommended Approach: Space Optimized.

Key Observation

`dp[i]` = minimum cost to reach step `i`. The top is step `n` (past the array). `dp[i] = cost[i] + min(dp[i-1], dp[i-2])`.

Python Code

Recursion:

```
def minCostClimbingStairs(cost):  
    n = len(cost)
```

```
def solve(i):
    if i < 2:
        return cost[i]
    return cost[i] + min(solve(i - 1), solve(i - 2))
return min(solve(n - 1), solve(n - 2))
```

Memoization:

```
def minCostClimbingStairs(cost):
    n = len(cost)
    memo = {}
    def solve(i):
        if i in memo:
            return memo[i]
        if i < 2:
            return cost[i]
        memo[i] = cost[i] + min(solve(i - 1), solve(i - 2))
        return memo[i]
    return min(solve(n - 1), solve(n - 2))
```

Tabulation:

```
def minCostClimbingStairs(cost):
    n = len(cost)
    dp = [0] * n
    dp[0], dp[1] = cost[0], cost[1]
    for i in range(2, n):
        dp[i] = cost[i] + min(dp[i - 1], dp[i - 2])
    return min(dp[n - 1], dp[n - 2])
```

Space Optimized:

```
def minCostClimbingStairs(cost):
    prev2, prev1 = cost[0], cost[1]
    for i in range(2, len(cost)):
        curr = cost[i] + min(prev1, prev2)
```

```
    prev2 = prev1
    prev1 = curr
    return min(prev1, prev2)
```

Complexity

- **Time:** $O(n)$
 - **Space:** $O(1)$
-

LC 91. Decode Ways

Difficulty: Medium **Pattern:** 1D Fibonacci Style

Problem

A message encoded as digits '1' to '26' maps to 'A' to 'Z'. Given a string of digits, return the **number of ways** to decode it. '0' alone is invalid. Leading zeros are invalid.

Approaches

Approach 1 — Recursion

- At each position, try decoding 1 digit and 2 digits (if valid).
- **Time:** $O(2^n)$ — **Space:** $O(n)$

Approach 2 — Memoization

- **Time:** $O(n)$ — **Space:** $O(n)$

Approach 3 — Tabulation

- **Time:** $O(n)$ — **Space:** $O(n)$

Approach 4 — Space Optimized

- **Time:** $O(n)$ — **Space:** $O(1)$

Recommended Approach: Space Optimized.

Key Observation

- If `s[i] != '0'`, one-digit decode is valid → add `dp[i-1]`.
- If `s[i-1:i+1]` forms 10–26, two-digit decode is valid → add `dp[i-2]`.
- If `s[i] == '0'` and no valid two-digit decode → 0 ways (impossible).

Python Code

Recursion:

```
def numDecodings(s):
    def solve(i):
        if i == len(s):
            return 1
        if s[i] == '0':
            return 0
        ways = solve(i + 1)
        if i + 1 < len(s) and int(s[i:i+2]) <= 26:
            ways += solve(i + 2)
        return ways
    return solve(0)
```

Memoization:

```
def numDecodings(s):
    memo = {}
    def solve(i):
        if i in memo:
            return memo[i]
        if i == len(s):
            return 1
        if s[i] == '0':
            return 0
        ways = solve(i + 1)
        if i + 1 < len(s) and int(s[i:i+2]) <= 26:
            ways += solve(i + 2)
        memo[i] = ways
    return solve(0)
```

```
    return ways
return solve(0)
```

Tabulation:

```
def numDecodings(s):
    n = len(s)
    dp = [0] * (n + 1)
    dp[n] = 1
    for i in range(n - 1, -1, -1):
        if s[i] == '0':
            dp[i] = 0
        else:
            dp[i] = dp[i + 1]
            if i + 1 < n and int(s[i:i+2]) <= 26:
                dp[i] += dp[i + 2]
    return dp[0]
```

Space Optimized:

```
def numDecodings(s):
    n = len(s)
    next1, next2 = 1, 0 # dp[n], dp[n+1]
    for i in range(n - 1, -1, -1):
        curr = 0
        if s[i] != '0':
            curr = next1
            if i + 1 < n and int(s[i:i+2]) <= 26:
                curr += next2
            next2 = next1
            next1 = curr
    return next1
```

Dry Run

Input: s = "226"

i	s[i]	1-digit valid?	2-digit valid?	dp[i]
2	'6'	Yes	—	1
1	'2'	Yes (dp[2]=1)	'26' ≤ 26 → Yes (dp[3]=1)	2
0	'2'	Yes (dp[1]=2)	'22' ≤ 26 → Yes (dp[2]=1)	3

Answer: 3 → "BZ", "VF", "BBF"

Complexity

- **Time:** $O(n)$
- **Space:** $O(1)$

LC 198. House Robber

Difficulty: Medium **Pattern:** 1D Fibonacci Style

Problem

Given an array of non-negative integers representing money in each house, you cannot rob **two adjacent houses**. Return the **maximum** amount you can rob.

Approaches

Highly important problem. Interviewers expect multiple approaches.

Approach 1 — Recursion

- At each house: rob it + skip next, or skip it.
- `rob(i) = max(nums[i] + rob(i+2), rob(i+1))`
- **Time:** $O(2^n)$ — **Space:** $O(n)$

Approach 2 — Memoization

- **Time:** $O(n)$ — **Space:** $O(n)$

Approach 3 — Tabulation

- `dp[i] = max(dp[i-1], dp[i-2] + nums[i])`
- **Time:** $O(n)$ — **Space:** $O(n)$

Approach 4 — Space Optimized

- **Time:** $O(n)$ — **Space:** $O(1)$

Recommended Approach: Space Optimized.

Key Observation

At each house i , two choices: 1. **Rob house i** $\rightarrow$ can't rob $i-1$, so add `dp[i-2] + nums[i]` 2. **Skip house i** $\rightarrow$ keep `dp[i-1]`

Take the maximum.

Python Code

Recursion:

```
def rob(nums):
    def solve(i):
        if i >= len(nums):
            return 0
        return max(nums[i] + solve(i + 2), solve(i + 1))
    return solve(0)
```

Memoization:

```
def rob(nums):
    memo = {}
    def solve(i):
        if i in memo:
            return memo[i]
```

```

        if i >= len(nums):
            return 0
        memo[i] = max(nums[i] + solve(i + 2), solve(i + 1))
        return memo[i]
    return solve(0)

```

Tabulation:

```

def rob(nums):
    n = len(nums)
    if n == 1:
        return nums[0]
    dp = [0] * n
    dp[0] = nums[0]
    dp[1] = max(nums[0], nums[1])
    for i in range(2, n):
        dp[i] = max(dp[i - 1], dp[i - 2] + nums[i])
    return dp[-1]

```

Space Optimized:

```

def rob(nums):
    prev2, prev1 = 0, 0
    for num in nums:
        curr = max(prev1, prev2 + num)
        prev2 = prev1
        prev1 = curr
    return prev1

```

Dry Run

Input: `nums = [2, 7, 9, 3, 1]`

i	num	prev2	prev1	curr = max(prev1, prev2+num)
0	2	0	2	2
1	7	2	7	7
2	9	7	9	9
3	3	9	9	12
4	1	9	12	12

0	2	0	0	$\max(0, 0+2)$ = 2
1	7	0	2	$\max(2, 0+7)$ = 7
2	9	2	7	$\max(7, 2+9)$ = 11
3	3	7	11	$\max(11, 7+3)$ = 11
4	1	11	11	$\max(11, 11+1)$ = 12

Answer: 12 (rob houses 0, 2, 4 $\rightarrow$ 2+9+1=12)

Complexity

- **Time:** $O(n)$
- **Space:** $O(1)$

LC 213. House Robber II

Difficulty: Medium **Pattern:** 1D Fibonacci Style

Problem

Same as House Robber, but houses are arranged in a **circle** — the first and last houses are adjacent. Cannot rob both first and last.

Approaches

Approach 1 — Recursion with Two Cases

- Case 1: Rob houses $[0, n-2]$ (exclude last).

- Case 2: Rob houses `[1, n-1]` (exclude first).
- Answer = max of both cases.
- **Time:** $O(2^n)$ — **Space:** $O(n)$

Approach 2 — Memoization (two calls)

- **Time:** $O(n)$ — **Space:** $O(n)$

Approach 3 — Tabulation (two passes of House Robber I)

- **Time:** $O(n)$ — **Space:** $O(1)$

Recommended Approach: Two passes of space-optimized House Robber I.

Key Observation

Break the circular constraint: if you rob house 0, you can't rob house $n-1$, and vice versa. So run House Robber I twice on `[0..n-2]` and `[1..n-1]`.

Python Code

Recursion:

```
def rob(nums):
    if len(nums) == 1:
        return nums[0]
    def solve(arr, i):
        if i >= len(arr):
            return 0
        return max(arr[i] + solve(arr, i + 2), solve(arr, i + 1))
    return max(solve(nums[:-1], 0), solve(nums[1:], 0))
```

Memoization:

```
def rob(nums):
    if len(nums) == 1:
        return nums[0]
    def solve(arr, i, memo):
```

```

        if i in memo:
            return memo[i]
        if i >= len(arr):
            return 0
        memo[i] = max(arr[i] + solve(arr, i + 2, memo), solve(arr, i + 1, memo))
        return memo[i]
    return max(solve(nums[:-1], 0, {}), solve(nums[1:], 0, {}))

```

Tabulation (Space Optimized):

```

def rob(nums):
    if len(nums) == 1:
        return nums[0]

    def rob_linear(arr):
        prev2, prev1 = 0, 0
        for num in arr:
            curr = max(prev1, prev2 + num)
            prev2 = prev1
            prev1 = curr
        return prev1

    return max(rob_linear(nums[:-1]), rob_linear(nums[1:]))

```

Complexity

- **Time:** $O(n)$
- **Space:** $O(1)$

LC 337. House Robber III

Difficulty: Medium **Pattern:** 1D Fibonacci Style (on Trees)

Problem

Houses are arranged in a **binary tree**. The thief cannot rob two directly-linked (parent-child) nodes. Return the **maximum** amount.

Approaches

Approach 1 — Recursion (Brute Force)

- For each node: rob it (skip children, go to grandchildren) or skip it (take children).
- **Time:** $O(2^n)$ — **Space:** $O(n)$

Approach 2 — Memoization

- Cache results per node.
- **Time:** $O(n)$ — **Space:** $O(n)$

Approach 3 — Optimal: Return Pair (rob, not_rob)

- Each node returns (rob_this, skip_this).
- `rob_this = node.val + left_skip + right_skip`
- `skip_this = max(left_rob, left_skip) + max(right_rob, right_skip)`
- Single DFS, no extra storage.
- **Time:** $O(n)$ — **Space:** $O(h)$ where h = tree height

Recommended Approach: Return pair.

Python Code

Recursion:

```
def rob(root):
    if not root:
        return 0
    rob_this = root.val
    if root.left:
        rob_this += rob(root.left.left) + rob(root.left.right)
    if root.right:
        rob_this += rob(root.right.left) + rob(root.right.right)
    skip_this = rob(root.left) + rob(root.right)
    return max(rob_this, skip_this)
```

Memoization:

```
def rob(root):
    memo = {}
    def solve(node):
        if not node:
            return 0
        if id(node) in memo:
            return memo[id(node)]
        rob_this = node.val
        if node.left:
            rob_this += solve(node.left.left) + solve(node.left.right)
        if node.right:
            rob_this += solve(node.right.left) + solve(node.right.right)
        skip_this = solve(node.left) + solve(node.right)
        memo[id(node)] = max(rob_this, skip_this)
        return memo[id(node)]
    return solve(root)
```

Tabulation-style (Pair DFS — Optimal):

```
def rob(root):
    def dfs(node):
        if not node:
            return (0, 0) # (rob, skip)
        left = dfs(node.left)
        right = dfs(node.right)
        rob_this = node.val + left[1] + right[1]
        skip_this = max(left) + max(right)
        return (rob_this, skip_this)
    return max(dfs(root))
```

Complexity

- **Time:** $O(n)$ — visit each node once.
 - **Space:** $O(h)$ — recursion stack, h = tree height.
-

LC 740. Delete and Earn

Difficulty: Medium **Pattern:** 1D Fibonacci Style

Problem

Given an array `nums`, when you pick a number `x`, you earn `x` points but must **delete all occurrences of `x-1` and `x+1`**. Return the **maximum** points.

Approaches

Approach 1 — Recursion

- Transform to House Robber: create `points[x] = x * count(x)`.
- Adjacent values can't both be picked → same as House Robber on `points`.
- **Time:** $O(2^{\text{max_val}})$ — **Space:** $O(\text{max_val})$

Approach 2 — Memoization

- **Time:** $O(\text{max_val})$ — **Space:** $O(\text{max_val})$

Approach 3 — Tabulation (Space Optimized)

- **Time:** $O(\text{max_val} + n)$ — **Space:** $O(\text{max_val})$

Recommended Approach: Transform to House Robber, then apply space-optimized solution.

Key Observation

- Compute `points[val] = val * frequency` for each value.
- Now the problem becomes: don't pick adjacent indices → House Robber on `points[0..max_val]`.

Python Code

Recursion:

```

def deleteAndEarn(nums):
    max_val = max(nums)
    points = [0] * (max_val + 1)
    for num in nums:
        points[num] += num

    def solve(i):
        if i < 0:
            return 0
        return max(solve(i - 1), solve(i - 2) + points[i])
    return solve(max_val)

```

Memoization:

```

def deleteAndEarn(nums):
    max_val = max(nums)
    points = [0] * (max_val + 1)
    for num in nums:
        points[num] += num
    memo = {}

    def solve(i):
        if i in memo:
            return memo[i]
        if i < 0:
            return 0
        memo[i] = max(solve(i - 1), solve(i - 2) + points[i])
        return memo[i]
    return solve(max_val)

```

Tabulation (Space Optimized):

```

def deleteAndEarn(nums):
    max_val = max(nums)
    points = [0] * (max_val + 1)
    for num in nums:

```

```
        points[num] += num
    }

    prev2, prev1 = 0, 0
    for i in range(max_val + 1):
        curr = max(prev1, prev2 + points[i])
        prev2 = prev1
        prev1 = curr
    return prev1
```

Complexity

- **Time:** $O(n + \text{max_val})$
- **Space:** $O(\text{max_val})$

Pattern 28: DP - 1D Array (Kadane's Algorithm for Max/Min Subarray)

LC 53. Maximum Subarray

Difficulty: Medium **Pattern:** Kadane's Algorithm

Problem

Given an integer array `nums`, find the **contiguous subarray** with the largest sum and return the sum.

Approaches

This is one of the most frequently asked interview problems. Know all approaches.

Approach 1 — Brute Force

- Try all subarrays, compute sum.
- **Time:** $O(n^2)$ — **Space:** $O(1)$

Approach 2 — Kadane's Algorithm (Optimal)

- Maintain running sum. At each element: extend current subarray or start new.
- `max_ending_here = max(nums[i], max_ending_here + nums[i])`
- **Time:** $O(n)$ — **Space:** $O(1)$

Approach 3 — Divide and Conquer

- Split array, find max subarray in left half, right half, and crossing middle.
- **Time:** $O(n \log n)$ — **Space:** $O(\log n)$
- Not optimal but good to know for interviews.

Recommended Approach: Kadane's Algorithm.

Kadane's Algorithm — Explanation

Key insight: At each index, you have two choices: 1. Extend the previous subarray by adding `nums[i]`. 2. Start a new subarray from `nums[i]`.

Choose the larger option. Track the global maximum across all positions.

Why it works: If `max_ending_here` becomes negative, starting fresh is always better.

Algorithm

1. Initialize `max_ending_here = nums[0]`, `max_so_far = nums[0]`.
2. For each element from index 1: - `max_ending_here = max(nums[i], max_ending_here + nums[i])` - `max_so_far = max(max_so_far, max_ending_here)`
3. Return `max_so_far`.

Python Code

Brute Force:

```
def maxSubArray(nums):
    max_sum = float('-inf')
    for i in range(len(nums)):
        curr = 0
        for j in range(i, len(nums)):
```

```
        curr += nums[j]
        max_sum = max(max_sum, curr)
    return max_sum
```

Kadane's (Optimal) — DP View:

```
def maxSubArray(nums):
    max_ending_here = nums[0]
    max_so_far = nums[0]
    for i in range(1, len(nums)):
        max_ending_here = max(nums[i], max_ending_here + nums[i])
        max_so_far = max(max_so_far, max_ending_here)
    return max_so_far
```

Tabulation View (explicit dp array):

```
def maxSubArray(nums):
    n = len(nums)
    dp = [0] * n
    dp[0] = nums[0]
    for i in range(1, n):
        dp[i] = max(nums[i], dp[i - 1] + nums[i])
    return max(dp)
```

Dry Run

Input: `nums = [-2, 1, -3, 4, -1, 2, 1, -5, 4]`

i	nums[i]	max_ending_here	max_so_far
0	-2	-2	-2
1	1	$\max(1, -2+1) = 1$	1
2	-3	$\max(-3, 1-3) = -2$	1

3	4	$\max(4, -2+4) = 4$	4
4	-1	$\max(-1, 4-1) = 3$	4
5	2	$\max(2, 3+2) = 5$	5
6	1	$\max(1, 5+1) = 6$	6
7	-5	$\max(-5, 6-5) = 1$	6
8	4	$\max(4, 1+4) = 5$	6

Answer: 6 (subarray [4, -1, 2, 1])

Complexity

- **Time:** $O(n)$
- **Space:** $O(1)$

Pattern 29: DP - 1D Array (Coin Change / Unbounded Knapsack Style)

LC 322. Coin Change

Difficulty: Medium **Pattern:** Unbounded Knapsack / Coin Change

Problem

Given coin denominations and a target amount, return the **fewest coins** needed to make that amount. If impossible, return **-1**. You have unlimited supply of each coin.

Approaches

One of the most important DP problems. Know all approaches thoroughly.

Approach 1 — Recursion (Brute Force)

- Try every coin at each step, recurse on remaining amount.
- **Time:** $O(\text{amount}^n)$ — **Space:** $O(\text{amount})$

Approach 2 — Memoization

- Cache results for each amount.
- **Time:** $O(\text{amount} \times n)$ — **Space:** $O(\text{amount})$

Approach 3 — Tabulation (Bottom-Up)

- $\text{dp}[a] = \min(\text{dp}[a], \text{dp}[a - \text{coin}] + 1)$ for each coin.
- **Time:** $O(\text{amount} \times n)$ — **Space:** $O(\text{amount})$

Recommended Approach: Tabulation.

Key Observation

- $\text{dp}[a]$ = minimum coins to make amount a .
- For each coin, if $a - \text{coin} \geq 0$ and $\text{dp}[a - \text{coin}]$ is reachable, then $\text{dp}[a] = \min(\text{dp}[a], \text{dp}[a - \text{coin}] + 1)$.
- Initialize $\text{dp}[0] = 0$, all others $\text{float}('inf')$.

Algorithm

1. Create dp of size $\text{amount} + 1$, fill with $\text{float}('inf')$.
2. Set $\text{dp}[0] = 0$.
3. For each amount from 1 to amount : - For each coin, if $\text{coin} \leq a$, update $\text{dp}[a] = \min(\text{dp}[a], \text{dp}[a - \text{coin}] + 1)$.
4. Return $\text{dp}[\text{amount}]$ if not infinity, else -1 .

Python Code

Recursion:

```
def coinChange(coins, amount):
    def solve(a):
        if a == 0:
            return 0
        if a < 0:
            return float('inf')
        res = float('inf')
        for coin in coins:
            res = min(res, solve(a - coin) + 1)
        return res
    ans = solve(amount)
    return ans if ans != float('inf') else -1
```

Memoization:

```
def coinChange(coins, amount):
    memo = {}
    def solve(a):
        if a in memo:
            return memo[a]
        if a == 0:
            return 0
        if a < 0:
            return float('inf')
        res = float('inf')
        for coin in coins:
            res = min(res, solve(a - coin) + 1)
        memo[a] = res
        return res
    ans = solve(amount)
    return ans if ans != float('inf') else -1
```

Tabulation:

```
def coinChange(coins, amount):
    dp = [float('inf')] * (amount + 1)
```

```

dp[0] = 0
for a in range(1, amount + 1):
    for coin in coins:
        if coin <= a:
            dp[a] = min(dp[a], dp[a - coin] + 1)
return dp[amount] if dp[amount] != float('inf') else -1

```

Dry Run

Input: coins = [1, 3, 4], amount = 6

a	Try coin 1	Try coin 3	Try coin 4	dp[a]
0	—	—	—	0
1	dp[0]+1=1	—	—	1
2	dp[1]+1=2	—	—	2
3	dp[2]+1=3	dp[0]+1=1	—	1
4	dp[3]+1=2	dp[1]+1=2	dp[0]+1=1	1
5	dp[4]+1=2	dp[2]+1=3	dp[1]+1=2	2
6	dp[5]+1=3	dp[3]+1=2	dp[2]+1=3	2

Answer: 2 (coins 3 + 3)

Complexity

- **Time:** $O(\text{amount} \times \text{len}(\text{coins}))$
 - **Space:** $O(\text{amount})$
-

LC 377. Combination Sum IV

Difficulty: Medium **Pattern:** Unbounded Knapsack (Permutations)

Problem

Given an array of distinct integers `nums` and a target, return the **number of possible combinations** (order matters, so actually **permutations**) that add up to target.

Approaches

Approach 1 — Recursion

- At each step, try every number that doesn't exceed remaining target.
- **Time:** Exponential — **Space:** $O(\text{target})$

Approach 2 — Memoization

- **Time:** $O(\text{target} \times n)$ — **Space:** $O(\text{target})$

Approach 3 — Tabulation

- **Time:** $O(\text{target} \times n)$ — **Space:** $O(\text{target})$

Recommended Approach: Tabulation.

Key Observation

Order matters → this counts permutations. Loop structure: - Outer loop: amounts (1 to target). - Inner loop: `nums`.

This is the **opposite** of Coin Change II where order doesn't matter.

Python Code

Recursion:

```
def combinationSum4(nums, target):
    def solve(t):
        if t == 0:
            return 1
```

```

        res = 0
        for num in nums:
            if num <= t:
                res += solve(t - num)
        return res
    return solve(target)

```

Memoization:

```

def combinationSum4(nums, target):
    memo = {}
    def solve(t):
        if t in memo:
            return memo[t]
        if t == 0:
            return 1
        res = 0
        for num in nums:
            if num <= t:
                res += solve(t - num)
        memo[t] = res
        return res
    return solve(target)

```

Tabulation:

```

def combinationSum4(nums, target):
    dp = [0] * (target + 1)
    dp[0] = 1
    for t in range(1, target + 1):
        for num in nums:
            if num <= t:
                dp[t] += dp[t - num]
    return dp[target]

```

Complexity

- **Time:** $O(\text{target} \times n)$
 - **Space:** $O(\text{target})$
-

LC 518. Coin Change II

Difficulty: Medium **Pattern:** Unbounded Knapsack (Combinations)

Problem

Given coin denominations and an amount, return the **number of combinations** (order doesn't matter) that make up that amount. Unlimited supply of each coin.

Approaches

Approach 1 — Recursion

- For each coin, either include it (stay on same coin) or move to next coin.
- **Time:** Exponential — **Space:** $O(\text{amount} + n)$

Approach 2 — Memoization

- State: `(coin_index, remaining_amount)`.
- **Time:** $O(n \times \text{amount})$ — **Space:** $O(n \times \text{amount})$

Approach 3 — Tabulation

- **Time:** $O(n \times \text{amount})$ — **Space:** $O(\text{amount})$

Recommended Approach: Tabulation.

Key Observation

Order doesn't matter → count combinations. Loop structure: - **Outer loop: coins** (process each coin fully before next). - **Inner loop: amounts** (left to right, since unbounded).

This ensures `[1, 2]` and `[2, 1]` are counted as the same combination.

Critical difference from Combination Sum IV:

Problem	Counts	Outer Loop	Inner Loop
Coin Change II	Combinations	Coins	Amounts
Combination Sum IV	Permutations	Amounts	Nums

Python Code

Recursion:

```
def change(amount, coins):
    def solve(i, a):
        if a == 0:
            return 1
        if i == len(coins) or a < 0:
            return 0
        return solve(i, a - coins[i]) + solve(i + 1, a)
    return solve(0, amount)
```

Memoization:

```
def change(amount, coins):
    memo = {}
    def solve(i, a):
        if (i, a) in memo:
            return memo[(i, a)]
        if a == 0:
            return 1
        if i == len(coins) or a < 0:
            return 0
        memo[(i, a)] = solve(i, a - coins[i]) + solve(i + 1, a)
        return memo[(i, a)]
    return solve(0, amount)
```

Tabulation:

```
def change(amount, coins):
    dp = [0] * (amount + 1)
    dp[0] = 1
    for coin in coins:          # outer: coins
        for a in range(coin, amount + 1): # inner: amounts (left to right)
            dp[a] += dp[a - coin]
    return dp[amount]
```

Dry Run

Input: `coins = [1, 2, 5]`, `amount = 5`

After processing coin=1: `dp = [1, 1, 1, 1, 1, 1]` After processing coin=2: `dp = [1, 1, 2, 2, 3, 3]` After processing coin=5: `dp = [1, 1, 2, 2, 3, 4]`

Answer: 4 → {5}, {2+2+1}, {2+1+1+1}, {1+1+1+1+1}

Complexity

- **Time:** $O(n \times \text{amount})$
- **Space:** $O(\text{amount})$

Pattern 30: DP - 1D Array (0/1 Knapsack Subset Sum Style)

LC 416. Partition Equal Subset Sum

Difficulty: Medium **Pattern:** 0/1 Knapsack / Subset Sum

Problem

Given a non-empty integer array `nums`, determine if it can be partitioned into **two subsets** with **equal sum**.

Approaches

Highly important. Classic 0/1 Knapsack reduction.

Approach 1 — Recursion

- Total must be even. $\text{Target} = \text{total} / 2$.
- For each number: include or exclude. Check if any subset sums to target.
- **Time:** $O(2^n)$ — **Space:** $O(n)$

Approach 2 — Memoization

- State: `(index, remaining_sum)`.
- **Time:** $O(n \times \text{target})$ — **Space:** $O(n \times \text{target})$

Approach 3 — Tabulation (1D)

- `dp[j]` = can we make sum `j`?
- For each num, iterate `j` from target down to num (right to left for 0/1 knapsack).
- **Time:** $O(n \times \text{target})$ — **Space:** $O(\text{target})$

Recommended Approach: 1D Tabulation.

Key Observation

- If total is odd $\rightarrow$ impossible.
- Reduces to: "Can we find a subset with sum = total/2?"
- This is a **subset sum** problem, a special case of **0/1 Knapsack**.
- **Must iterate right-to-left** in 1D DP so each number is used at most once.

Algorithm

1. Compute total. If odd, return `False`.
2. `target = total // 2`.
3. `dp = [False] * (target + 1)`, `dp[0] = True`.
4. For each num in nums: - For `j` from target down to num: `dp[j] = dp[j] or dp[j - num]`.
5. Return `dp[target]`.

Python Code

Recursion:

```
def canPartition(nums):
    total = sum(nums)
    if total % 2:
        return False
    target = total // 2

    def solve(i, remaining):
        if remaining == 0:
            return True
        if i >= len(nums) or remaining < 0:
            return False
        return solve(i + 1, remaining - nums[i]) or solve(i + 1, remaining)
    return solve(0, target)
```

Memoization:

```
def canPartition(nums):
    total = sum(nums)
    if total % 2:
        return False
    target = total // 2
    memo = {}

    def solve(i, remaining):
        if (i, remaining) in memo:
            return memo[(i, remaining)]
        if remaining == 0:
            return True
        if i >= len(nums) or remaining < 0:
            return False
        memo[(i, remaining)] = solve(i + 1, remaining - nums[i]) or solve(i + 1, remaining)
        return memo[(i, remaining)]
    return solve(0, target)
```

Tabulation:

```
def canPartition(nums):
    total = sum(nums)
    if total % 2:
        return False
    target = total // 2
    dp = [False] * (target + 1)
    dp[0] = True
    for num in nums:
        for j in range(target, num - 1, -1): # right to left!
            dp[j] = dp[j] or dp[j - num]
    return dp[target]
```

Dry Run

Input: `nums = [1, 5, 11, 5]`, `total = 22`, `target = 11`

num	dp (indices 0-11 that become True)
init	{0}
1	{0, 1}
5	{0, 1, 5, 6}
11	{0, 1, 5, 6, 11} ← target reached!

Answer: True (subset {1, 5, 5} and {11})

Complexity

- **Time:** $O(n \times \text{target})$
- **Space:** $O(\text{target})$

LC 494. Target Sum

Difficulty: Medium **Pattern:** 0/1 Knapsack / Subset Sum (Count)

Problem

Given an array `nums` and a target, assign `+` or `-` to each number. Return the **number of ways** to reach the target sum.

Approaches

Approach 1 — Recursion

- At each index, add or subtract the number.
- **Time:** $O(2^n)$ — **Space:** $O(n)$

Approach 2 — Memoization

- State: `(index, current_sum)`.
- **Time:** $O(n \times \text{total_sum})$ — **Space:** $O(n \times \text{total_sum})$

Approach 3 — Tabulation (Subset Sum Reduction)

- **Time:** $O(n \times \text{target_subset})$ — **Space:** $O(\text{target_subset})$

Recommended Approach: Subset Sum Reduction + 1D DP.

Key Observation (Mathematical Reduction)

Let P = set of numbers with `+`, N = set with `-`.

$$\begin{aligned} P - N &= \text{target} \\ P + N &= \text{total} \\ \rightarrow 2P &= \text{target} + \text{total} \\ \rightarrow P &= (\text{target} + \text{total}) / 2 \end{aligned}$$

Problem reduces to: **count subsets that sum to P** . This is 0/1 Knapsack counting.

Validity checks: - `(target + total)` must be even. - `abs(target)` must be $\leq$ `total`.

Python Code

Recursion:

```
def findTargetSumWays(nums, target):  
    def solve(i, curr_sum):  
        if i == len(nums):  
            return 1 if curr_sum == target else 0  
        return solve(i + 1, curr_sum + nums[i]) + solve(i + 1, curr_sum - nums[i])  
    return solve(0, 0)
```

Memoization:

```
def findTargetSumWays(nums, target):  
    memo = {}  
    def solve(i, curr_sum):  
        if (i, curr_sum) in memo:  
            return memo[(i, curr_sum)]  
        if i == len(nums):  
            return 1 if curr_sum == target else 0  
        memo[(i, curr_sum)] = solve(i + 1, curr_sum + nums[i]) + solve(i + 1, curr_sum - nums[i])  
        return memo[(i, curr_sum)]  
    return solve(0, 0)
```

Tabulation (Subset Sum Reduction):

```
def findTargetSumWays(nums, target):  
    total = sum(nums)  
    if (target + total) % 2 or abs(target) > total:  
        return 0  
    subset_target = (target + total) // 2  
  
    dp = [0] * (subset_target + 1)  
    dp[0] = 1  
    for num in nums:  
        for j in range(subset_target, num - 1, -1): # right to left  
            dp[j] += dp[j - num]  
    return dp[subset_target]
```

Complexity

- **Time:** $O(n \times \text{subset_target})$
 - **Space:** $O(\text{subset_target})$
-

Pattern 31: DP - 1D Array (Word Break Style)

LC 139. Word Break

Difficulty: Medium **Pattern:** Word Break

Problem

Given a string `s` and a dictionary `wordDict`, return `True` if `s` can be segmented into a space-separated sequence of dictionary words. Words can be reused.

Approaches

Approach 1 — Recursion

- Try every prefix. If prefix is in dictionary, recurse on suffix.
- **Time:** $O(2^n)$ — **Space:** $O(n)$

Approach 2 — Memoization

- Cache `canBreak(i)` = can `s[i:]` be broken?
- **Time:** $O(n^2 \times m)$ where m = max word length — **Space:** $O(n)$

Approach 3 — Tabulation

- `dp[i]` = can `s[0:i]` be segmented?
- **Time:** $O(n^2 \times m)$ — **Space:** $O(n)$

Recommended Approach: Tabulation.

Key Observation

`dp[i] = True` if there exists some `j < i` such that `dp[j] == True` and `s[j:i]` is in `wordDict`.

Optimization: only check substrings up to the length of the longest word in the dictionary.

Algorithm

1. Convert `wordDict` to a set for $O(1)$ lookup.
2. `dp = [False] * (n + 1)`, `dp[0] = True`.
3. For `i` from 1 to `n`: - For `j` from 0 to `i`: if `dp[j]` and `s[j:i]` in `wordSet` → `dp[i] = True`, break.
4. Return `dp[n]`.

Python Code

Recursion:

```
def wordBreak(s, wordDict):
    wordSet = set(wordDict)
    def solve(i):
        if i == len(s):
            return True
        for j in range(i + 1, len(s) + 1):
            if s[i:j] in wordSet and solve(j):
                return True
        return False
    return solve(0)
```

Memoization:

```
def wordBreak(s, wordDict):
    wordSet = set(wordDict)
    memo = {}
    def solve(i):
        if i in memo:
            return memo[i]
        if i == len(s):
```

```

        return True
    for j in range(i + 1, len(s) + 1):
        if s[i:j] in wordSet and solve(j):
            memo[i] = True
            return True
    memo[i] = False
    return False
return solve(0)

```

Tabulation:

```

def wordBreak(s, wordDict):
    wordSet = set(wordDict)
    n = len(s)
    dp = [False] * (n + 1)
    dp[0] = True
    for i in range(1, n + 1):
        for j in range(i):
            if dp[j] and s[j:i] in wordSet:
                dp[i] = True
                break
    return dp[n]

```

Complexity

- **Time:** $O(n^2 \times m)$ where m = average word length (for substring hashing)
- **Space:** $O(n)$

LC 140. Word Break II

Difficulty: Hard **Pattern:** Word Break + Backtracking

Problem

Given a string `s` and a dictionary `wordDict`, return **all possible sentences** by segmenting `s` into dictionary words.

Approaches

Approach 1 — Recursion with Backtracking

- Try every prefix. If valid, recurse on suffix, collect all results.
- **Time:** $O(2^n \times n)$ — **Space:** $O(2^n \times n)$ for results

Approach 2 — Memoization with Backtracking

- Cache all valid segmentations from index i onward.
- **Time:** $O(2^n \times n)$ worst case — **Space:** $O(2^n \times n)$

Recommended Approach: Memoized backtracking.

Key Observation

Unlike Word Break I, we need **all** valid segmentations, not just True/False. Use recursion + memoization. Each position returns a list of all valid sentences from that position.

Python Code

Recursion:

```
def wordBreak(s, wordDict):
    wordSet = set(wordDict)
    def solve(i):
        if i == len(s):
            return [""]
        results = []
        for j in range(i + 1, len(s) + 1):
            word = s[i:j]
            if word in wordSet:
                for rest in solve(j):
                    if rest:
                        results.append(word + " " + rest)
                    else:
                        results.append(word)
        return results
    return solve(0)
```

Memoization:

```
def wordBreak(s, wordDict):
    wordSet = set(wordDict)
    memo = {}

    def solve(i):
        if i in memo:
            return memo[i]
        if i == len(s):
            return [""]
        results = []
        for j in range(i + 1, len(s) + 1):
            word = s[i:j]
            if word in wordSet:
                for rest in solve(j):
                    if rest:
                        results.append(word + " " + rest)
                    else:
                        results.append(word)
        memo[i] = results
        return results
    return solve(0)
```

Complexity

- **Time:** $O(2^n \times n)$ worst case (exponential number of valid sentences possible)
- **Space:** $O(2^n \times n)$ for storing all results

Pattern 32: DP - 2D Array (Longest Common Subsequence - LCS)

LC 1143. Longest Common Subsequence

Difficulty: Medium **Pattern:** LCS

Problem

Given two strings `text1` and `text2`, return the length of their **longest common subsequence**. A subsequence preserves order but need not be contiguous.

Approaches

Extremely important interview problem. Know all three approaches.

Approach 1 — Recursion

- Compare characters from the end. If match: $1 + \text{recurse}(i-1, j-1)$. If not: $\max(\text{recurse}(i-1, j), \text{recurse}(i, j-1))$.
- **Time:** $O(2^{m+n})$ — **Space:** $O(m+n)$

Approach 2 — Memoization

- State: (i, j) .
- **Time:** $O(m \times n)$ — **Space:** $O(m \times n)$

Approach 3 — Tabulation

- 2D DP table.
- **Time:** $O(m \times n)$ — **Space:** $O(m \times n)$, can optimize to $O(n)$

Recommended Approach: Tabulation.

Key Observation

```
If text1[i-1] == text2[j-1]:
    dp[i][j] = 1 + dp[i-1][j-1]    # match, extend LCS
Else:
    dp[i][j] = max(dp[i-1][j], dp[i][j-1]) # skip one character
```

Python Code

Recursion:

```
def longestCommonSubsequence(text1, text2):
    def solve(i, j):
        if i == 0 or j == 0:
            return 0
        if text1[i - 1] == text2[j - 1]:
            return 1 + solve(i - 1, j - 1)
        return max(solve(i - 1, j), solve(i, j - 1))
    return solve(len(text1), len(text2))
```

Memoization:

```
def longestCommonSubsequence(text1, text2):
    memo = {}
    def solve(i, j):
        if (i, j) in memo:
            return memo[(i, j)]
        if i == 0 or j == 0:
            return 0
        if text1[i - 1] == text2[j - 1]:
            memo[(i, j)] = 1 + solve(i - 1, j - 1)
        else:
            memo[(i, j)] = max(solve(i - 1, j), solve(i, j - 1))
        return memo[(i, j)]
    return solve(len(text1), len(text2))
```

Tabulation:

```
def longestCommonSubsequence(text1, text2):
    m, n = len(text1), len(text2)
    dp = [[0] * (n + 1) for _ in range(m + 1)]
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if text1[i - 1] == text2[j - 1]:
                dp[i][j] = 1 + dp[i - 1][j - 1]
            else:
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1])
```

```
return dp[m][n]
```

Space Optimized (1D):

```
def longestCommonSubsequence(text1, text2):
    m, n = len(text1), len(text2)
    prev = [0] * (n + 1)
    for i in range(1, m + 1):
        curr = [0] * (n + 1)
        for j in range(1, n + 1):
            if text1[i - 1] == text2[j - 1]:
                curr[j] = 1 + prev[j - 1]
            else:
                curr[j] = max(prev[j], curr[j - 1])
        prev = curr
    return prev[n]
```

Dry Run

Input: text1 = "abcde", text2 = "ace"

	""	a	c	e
"" 0	0	0	0	0
a 0		1	1	1
b 0		1	1	1
c 0		1	2	2
d 0		1	2	2
e 0		1	2	3

Answer: 3 (LCS = "ace")

Complexity

- **Time:** $O(m \times n)$
 - **Space:** $O(n)$ (space optimized)
-

LC 583. Delete Operation for Two Strings

Difficulty: Medium **Pattern:** LCS

Problem

Given two strings `word1` and `word2`, return the **minimum number of deletions** to make them equal.

Key Observation

After deletions, what remains is the **LCS**. So: $\text{answer} = \text{len}(\text{word1}) + \text{len}(\text{word2}) - 2 * \text{LCS}(\text{word1}, \text{word2})$.

Approaches

Approach 1 — Recursion (LCS-based)

- Find LCS recursively, compute deletions.
- **Time:** $O(2^{(m+n)})$ — **Space:** $O(m+n)$

Approach 2 — Memoization

- **Time:** $O(m \times n)$ — **Space:** $O(m \times n)$

Approach 3 — Tabulation

- **Time:** $O(m \times n)$ — **Space:** $O(n)$

Recommended Approach: LCS tabulation + formula.

Python Code

Recursion:

```
def minDistance(word1, word2):  
    def lcs(i, j):  
        if i == 0 or j == 0:  
            return 0  
        if word1[i - 1] == word2[j - 1]:  
            return 1 + lcs(i - 1, j - 1)  
        return max(lcs(i - 1, j), lcs(i, j - 1))  
    l = lcs(len(word1), len(word2))  
    return len(word1) + len(word2) - 2 * l
```

Memoization:

```
def minDistance(word1, word2):  
    memo = {}  
    def lcs(i, j):  
        if (i, j) in memo:  
            return memo[(i, j)]  
        if i == 0 or j == 0:  
            return 0  
        if word1[i - 1] == word2[j - 1]:  
            memo[(i, j)] = 1 + lcs(i - 1, j - 1)  
        else:  
            memo[(i, j)] = max(lcs(i - 1, j), lcs(i, j - 1))  
        return memo[(i, j)]  
    l = lcs(len(word1), len(word2))  
    return len(word1) + len(word2) - 2 * l
```

Tabulation:

```
def minDistance(word1, word2):  
    m, n = len(word1), len(word2)  
    prev = [0] * (n + 1)  
    for i in range(1, m + 1):  
        curr = [0] * (n + 1)
```

```
    for j in range(1, n + 1):
        if word1[i - 1] == word2[j - 1]:
            curr[j] = 1 + prev[j - 1]
        else:
            curr[j] = max(prev[j], curr[j - 1])
        prev = curr
    return m + n - 2 * prev[n]
```

Complexity

- **Time:** $O(m \times n)$
- **Space:** $O(n)$

Pattern 33: DP - 2D Array (Edit Distance / Levenshtein Distance)

LC 72. Edit Distance

Difficulty: Hard **Pattern:** Edit Distance

Problem

Given two strings `word1` and `word2`, return the **minimum number of operations** (insert, delete, replace) to convert `word1` to `word2`.

Approaches

One of the most important DP problems. Must know thoroughly.

Approach 1 — Recursion

- Compare characters from end. If match: move both. If not: try all 3 operations.
- **Time:** $O(3^{(m+n)})$ — **Space:** $O(m+n)$

Approach 2 — Memoization

- State: (i, j) .
- **Time:** $O(m \times n)$ — **Space:** $O(m \times n)$

Approach 3 — Tabulation

- **Time:** $O(m \times n)$ — **Space:** $O(m \times n)$, can optimize to $O(n)$

Recommended Approach: Tabulation.

Key Observation

```
If word1[i-1] == word2[j-1]:
    dp[i][j] = dp[i-1][j-1]          # no operation needed
Else:
    dp[i][j] = 1 + min(
        dp[i-1][j],      # delete from word1
        dp[i][j-1],      # insert into word1
        dp[i-1][j-1]     # replace in word1
    )
```

Base cases: - $dp[i][0] = i$ (delete all characters of word1) - $dp[0][j] = j$ (insert all characters of word2)

Algorithm

1. Create dp of size $(m+1) \times (n+1)$.
2. Fill base cases: first row and first column.
3. Fill table: match $\rightarrow$ diagonal, else $1 + \min(\text{up, left, diagonal})$.
4. Return $dp[m][n]$.

Python Code

Recursion:

```
def minDistance(word1, word2):
    def solve(i, j):
        if i == 0:
```

```

        return j
    if j == 0:
        return i
    if word1[i - 1] == word2[j - 1]:
        return solve(i - 1, j - 1)
    return 1 + min(solve(i - 1, j), solve(i, j - 1), solve(i - 1, j - 1))
return solve(len(word1), len(word2))

```

Memoization:

```

def minDistance(word1, word2):
    memo = {}
    def solve(i, j):
        if (i, j) in memo:
            return memo[(i, j)]
        if i == 0:
            return j
        if j == 0:
            return i
        if word1[i - 1] == word2[j - 1]:
            memo[(i, j)] = solve(i - 1, j - 1)
        else:
            memo[(i, j)] = 1 + min(solve(i - 1, j), solve(i, j - 1), solve(i - 1, j - 1))
        return memo[(i, j)]
    return solve(len(word1), len(word2))

```

Tabulation:

```

def minDistance(word1, word2):
    m, n = len(word1), len(word2)
    dp = [[0] * (n + 1) for _ in range(m + 1)]
    for i in range(m + 1):
        dp[i][0] = i
    for j in range(n + 1):
        dp[0][j] = j
    for i in range(1, m + 1):

```

```

        for j in range(1, n + 1):
            if word1[i - 1] == word2[j - 1]:
                dp[i][j] = dp[i - 1][j - 1]
            else:
                dp[i][j] = 1 + min(dp[i - 1][j], dp[i][j - 1], dp[i - 1][j])
        return dp[m][n]

```

Space Optimized:

```

def minDistance(word1, word2):
    m, n = len(word1), len(word2)
    prev = list(range(n + 1))
    for i in range(1, m + 1):
        curr = [i] + [0] * n
        for j in range(1, n + 1):
            if word1[i - 1] == word2[j - 1]:
                curr[j] = prev[j - 1]
            else:
                curr[j] = 1 + min(prev[j], curr[j - 1], prev[j - 1])
        prev = curr
    return prev[n]

```

Dry Run

Input: word1 = "horse", word2 = "ros"

		r	o	s
"" 0		1	2	3
h 1		1	2	3
o 2		2	1	2
r 3		2	2	2

s 4	3	3	2
e 5	4	4	3

Answer: 3 (horse → rorse → rose → ros)

Complexity

- **Time:** $O(m \times n)$
- **Space:** $O(n)$ (space optimized)

Pattern 34: DP - 2D Array (Unique Paths on Grid)

LC 62. Unique Paths

Difficulty: Medium **Pattern:** Grid DP

Problem

An $m \times n$ grid. Start at top-left, end at bottom-right. Can only move **right** or **down**. Count the number of **unique paths**.

Approaches

Approach 1 — Recursion

- At each cell: go right + go down.
- **Time:** $O(2^{(m+n)})$ — **Space:** $O(m+n)$

Approach 2 — Memoization

- **Time:** $O(m \times n)$ — **Space:** $O(m \times n)$

Approach 3 — Tabulation

- **Time:** $O(m \times n)$ — **Space:** $O(n)$

Recommended Approach: 1D Tabulation.

Key Observation

$dp[i][j] = dp[i-1][j] + dp[i][j-1]$. First row and first column are all 1s (only one way to reach each).

Python Code

Recursion:

```
def uniquePaths(m, n):
    def solve(i, j):
        if i == 0 or j == 0:
            return 1
        return solve(i - 1, j) + solve(i, j - 1)
    return solve(m - 1, n - 1)
```

Memoization:

```
def uniquePaths(m, n):
    memo = {}
    def solve(i, j):
        if (i, j) in memo:
            return memo[(i, j)]
        if i == 0 or j == 0:
            return 1
        memo[(i, j)] = solve(i - 1, j) + solve(i, j - 1)
        return memo[(i, j)]
    return solve(m - 1, n - 1)
```

Tabulation (1D Optimized):

```
def uniquePaths(m, n):
    dp = [1] * n
    for i in range(1, m):
        for j in range(1, n):
```

```
        dp[j] += dp[j - 1]
    return dp[-1]
```

Complexity

- **Time:** $O(m \times n)$
- **Space:** $O(n)$

LC 63. Unique Paths II

Difficulty: Medium **Pattern:** Grid DP

Problem

Same as Unique Paths, but the grid has **obstacles** (1 = obstacle, 0 = empty). Count unique paths avoiding obstacles.

Key Observation

Same recurrence, but if `grid[i][j] == 1`, then `dp[i][j] = 0`.

Python Code

Recursion:

```
def uniquePathsWithObstacles(grid):
    m, n = len(grid), len(grid[0])
    def solve(i, j):
        if i < 0 or j < 0 or grid[i][j] == 1:
            return 0
        if i == 0 and j == 0:
            return 1
        return solve(i - 1, j) + solve(i, j - 1)
    return solve(m - 1, n - 1)
```

Memoization:

```

def uniquePathsWithObstacles(grid):
    m, n = len(grid), len(grid[0])
    memo = {}
    def solve(i, j):
        if (i, j) in memo:
            return memo[(i, j)]
        if i < 0 or j < 0 or grid[i][j] == 1:
            return 0
        if i == 0 and j == 0:
            return 1
        memo[(i, j)] = solve(i - 1, j) + solve(i, j - 1)
        return memo[(i, j)]
    return solve(m - 1, n - 1)

```

Tabulation:

```

def uniquePathsWithObstacles(grid):
    m, n = len(grid), len(grid[0])
    if grid[0][0] == 1:
        return 0
    dp = [0] * n
    dp[0] = 1
    for i in range(m):
        for j in range(n):
            if grid[i][j] == 1:
                dp[j] = 0
            elif j > 0:
                dp[j] += dp[j - 1]
    return dp[-1]

```

Complexity

- **Time:** $O(m \times n)$
 - **Space:** $O(n)$
-

LC 64. Minimum Path Sum

Difficulty: Medium **Pattern:** Grid DP

Problem

Given an $m \times n$ grid of non-negative integers, find a path from top-left to bottom-right that **minimizes the sum**. Can only move right or down.

Key Observation

$dp[i][j] = grid[i][j] + \min(dp[i-1][j], dp[i][j-1])$.

Python Code

Recursion:

```
def minPathSum(grid):
    m, n = len(grid), len(grid[0])
    def solve(i, j):
        if i == 0 and j == 0:
            return grid[0][0]
        if i < 0 or j < 0:
            return float('inf')
        return grid[i][j] + min(solve(i - 1, j), solve(i, j - 1))
    return solve(m - 1, n - 1)
```

Memoization:

```
def minPathSum(grid):
    m, n = len(grid), len(grid[0])
    memo = {}
    def solve(i, j):
        if (i, j) in memo:
            return memo[(i, j)]
        if i == 0 and j == 0:
            return grid[0][0]
        if i < 0 or j < 0:
            return float('inf')
```

```

        memo[(i, j)] = grid[i][j] + min(solve(i - 1, j), solve(i, j - 1))
    return memo[(i, j)]
return solve(m - 1, n - 1)

```

Tabulation:

```

def minPathSum(grid):
    m, n = len(grid), len(grid[0])
    dp = [float('inf')] * n
    dp[0] = 0
    for i in range(m):
        dp[0] += grid[i][0]
        for j in range(1, n):
            if i == 0:
                dp[j] = dp[j - 1] + grid[i][j]
            else:
                dp[j] = grid[i][j] + min(dp[j], dp[j - 1])
    return dp[-1]

```

Cleaner Tabulation (2D):

```

def minPathSum(grid):
    m, n = len(grid), len(grid[0])
    dp = [[0] * n for _ in range(m)]
    dp[0][0] = grid[0][0]
    for i in range(1, m):
        dp[i][0] = dp[i - 1][0] + grid[i][0]
    for j in range(1, n):
        dp[0][j] = dp[0][j - 1] + grid[0][j]
    for i in range(1, m):
        for j in range(1, n):
            dp[i][j] = grid[i][j] + min(dp[i - 1][j], dp[i][j - 1])
    return dp[m - 1][n - 1]

```

Complexity

- **Time:** $O(m \times n)$

- **Space:** $O(n)$ (1D) or $O(m \times n)$ (2D)
-

LC 120. Triangle

Difficulty: Medium **Pattern:** Grid DP (Non-rectangular)

Problem

Given a triangle array, find the **minimum path sum** from top to bottom. At each step, you may move to adjacent numbers on the row below (index i or $i+1$).

Key Observation

Process **bottom-up**. $dp[j] = triangle[i][j] + \min(dp[j], dp[j+1])$. Starting from the last row, collapse upward.

Python Code

Recursion:

```
def minimumTotal(triangle):
    def solve(i, j):
        if i == len(triangle):
            return 0
        return triangle[i][j] + min(solve(i + 1, j), solve(i + 1, j + 1))
    return solve(0, 0)
```

Memoization:

```
def minimumTotal(triangle):
    memo = {}
    def solve(i, j):
        if (i, j) in memo:
            return memo[(i, j)]
        if i == len(triangle):
            return 0
        memo[(i, j)] = triangle[i][j] + min(solve(i + 1, j), solve(i + 1, j + 1))
        return memo[(i, j)]
```

```
return solve(0, 0)
```

Tabulation (Bottom-Up, $O(n)$ space):

```
def minimumTotal(triangle):  
    dp = triangle[-1][:]  
    for i in range(len(triangle) - 2, -1, -1):  
        for j in range(len(triangle[i])):  
            dp[j] = triangle[i][j] + min(dp[j], dp[j + 1])  
    return dp[0]
```

Complexity

- **Time:** $O(n^2)$ where n = number of rows
- **Space:** $O(n)$

LC 221. Maximal Square

Difficulty: Medium **Pattern:** Grid DP

Problem

Given an $m \times n$ binary matrix (containing '0' and '1'), find the **largest square** containing only '1's and return its **area**.

Approaches

Approach 1 — Brute Force

- For each cell, expand and check all possible squares.
- **Time:** $O(m \times n \times \min(m,n)^2)$ — **Space:** $O(1)$

Approach 2 — Tabulation (Optimal)

- $dp[i][j]$ = side length of largest square with bottom-right corner at (i, j) .
- **Time:** $O(m \times n)$ — **Space:** $O(n)$

Recommended Approach: Tabulation.

Key Observation

```
If matrix[i][j] == '1':
    dp[i][j] = min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1]) + 1
Else:
    dp[i][j] = 0
```

The side length is limited by the **smallest** of the three neighbors (top, left, top-left diagonal).

Python Code

Recursion:

```
def maximalSquare(matrix):
    m, n = len(matrix), len(matrix[0])
    res = 0
    def solve(i, j):
        nonlocal res
        if i < 0 or j < 0 or matrix[i][j] == '0':
            return 0
        right = solve(i, j - 1)
        below = solve(i - 1, j)
        diag = solve(i - 1, j - 1)
        side = min(right, below, diag) + 1
        res = max(res, side)
        return side
    # Need to call for each cell – not efficient as recursion
    # Better to use memoization or tabulation directly
    for i in range(m):
        for j in range(n):
            solve(i, j)
    return res * res
```

Memoization:

```

def maximalSquare(matrix):
    m, n = len(matrix), len(matrix[0])
    memo = {}
    res = 0

    def solve(i, j):
        if (i, j) in memo:
            return memo[(i, j)]
        if i < 0 or j < 0 or matrix[i][j] == '0':
            return 0
        memo[(i, j)] = min(solve(i - 1, j), solve(i, j - 1), solve(i - 1, j - 1)) + 1
        return memo[(i, j)]

    for i in range(m):
        for j in range(n):
            res = max(res, solve(i, j))
    return res * res

```

Tabulation:

```

def maximalSquare(matrix):
    m, n = len(matrix), len(matrix[0])
    dp = [0] * (n + 1)
    max_side = 0
    prev = 0
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            temp = dp[j]
            if matrix[i - 1][j - 1] == '1':
                dp[j] = min(dp[j], dp[j - 1], prev) + 1
                max_side = max(max_side, dp[j])
            else:
                dp[j] = 0
            prev = temp
    return max_side * max_side

```

Tabulation (2D — clearer):

```
def maximalSquare(matrix):
    m, n = len(matrix), len(matrix[0])
    dp = [[0] * (n + 1) for _ in range(m + 1)]
    max_side = 0
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if matrix[i - 1][j - 1] == '1':
                dp[i][j] = min(dp[i - 1][j], dp[i][j - 1], dp[i - 1][j - 1]) + 1
                max_side = max(max_side, dp[i][j])
    return max_side * max_side
```

Dry Run

Input:

```
1 0 1 0 0
1 0 1 1 1
1 1 1 1 1
1 0 0 1 0
```

DP table (relevant part):

```
0 0 0 0 0
0 0 0 0 0
0 0 0 1 1
0 0 0 1 2 ← max_side = 2
0 0 0 0 0
```

Answer: 4 (2×2 square)

Complexity

- **Time:** $O(m \times n)$
- **Space:** $O(n)$ (1D) or $O(m \times n)$ (2D)

LC 931. Minimum Falling Path Sum

Difficulty: Medium **Pattern:** Grid DP

Problem

Given an $n \times n$ matrix, find the **minimum sum** of a falling path. A falling path starts at any element in the first row and moves to the element directly below, below-left, or below-right.

Key Observation

$dp[i][j] = matrix[i][j] + \min(dp[i-1][j-1], dp[i-1][j], dp[i-1][j+1])$ with boundary handling. Answer = min of last row.

Python Code

Recursion:

```
def minFallingPathSum(matrix):
    n = len(matrix)
    def solve(i, j):
        if j < 0 or j >= n:
            return float('inf')
        if i == 0:
            return matrix[0][j]
        return matrix[i][j] + min(solve(i-1, j-1), solve(i-1, j), solve(i-1, j+1))
    return min(solve(n-1, j) for j in range(n))
```

Memoization:

```
def minFallingPathSum(matrix):
    n = len(matrix)
    memo = {}
    def solve(i, j):
        if (i, j) in memo:
            return memo[(i, j)]
        if j < 0 or j >= n:
            return float('inf')
        if i == 0:
            return matrix[0][j]
        return matrix[i][j] + min(solve(i-1, j-1), solve(i-1, j), solve(i-1, j+1))
    return min(solve(n-1, j) for j in range(n))
```

```

        memo[(i, j)] = matrix[i][j] + min(solve(i-1, j-1), solve(i-1, j), s
        return memo[(i, j)]
    return min(solve(n-1, j) for j in range(n))

```

Tabulation:

```

def minFallingPathSum(matrix):
    n = len(matrix)
    dp = matrix[0][:]
    for i in range(1, n):
        new_dp = [0] * n
        for j in range(n):
            left = dp[j - 1] if j > 0 else float('inf')
            mid = dp[j]
            right = dp[j + 1] if j < n - 1 else float('inf')
            new_dp[j] = matrix[i][j] + min(left, mid, right)
        dp = new_dp
    return min(dp)

```

Complexity

- **Time:** $O(n^2)$
- **Space:** $O(n)$

LC 1277. Count Square Submatrices with All Ones

Difficulty: Medium **Pattern:** Grid DP

Problem

Given an $m \times n$ binary matrix, return the **total number** of square submatrices that have all ones.

Key Observation

Uses the same DP as Maximal Square. $dp[i][j]$ = side length of largest all-1s square with bottom-right at (i, j) . If $dp[i][j] = k$, it contributes k squares (of sizes 1×1 , 2×2 ,

..., k×k). Sum all `dp[i][j]` values.

Python Code

Recursion:

```
def countSquares(matrix):
    m, n = len(matrix), len(matrix[0])
    memo = {}
    def solve(i, j):
        if (i, j) in memo:
            return memo[(i, j)]
        if i < 0 or j < 0 or matrix[i][j] == 0:
            return 0
        memo[(i, j)] = min(solve(i-1, j), solve(i, j-1), solve(i-1, j-1)) + 1
        return memo[(i, j)]
    total = 0
    for i in range(m):
        for j in range(n):
            total += solve(i, j)
    return total
```

Tabulation:

```
def countSquares(matrix):
    m, n = len(matrix), len(matrix[0])
    dp = [[0] * n for _ in range(m)]
    total = 0
    for i in range(m):
        for j in range(n):
            if matrix[i][j] == 1:
                if i == 0 or j == 0:
                    dp[i][j] = 1
                else:
                    dp[i][j] = min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1]) + 1
            total += dp[i][j]
    return total
```

Complexity

- **Time:** $O(m \times n)$
 - **Space:** $O(m \times n)$ — can optimize to $O(n)$ with 1D array
-

Pattern 35: DP - Interval DP

LC 312. Burst Balloons

Difficulty: Hard **Pattern:** Interval DP

Problem

Given n balloons with values in `nums`, bursting balloon i earns `nums[i-1] * nums[i] * nums[i+1]` coins. After bursting, neighbors become adjacent. Return the **maximum** coins collectible.

Approaches

Approach 1 — Recursion (Try every order)

- Try every permutation of bursting order.
- **Time:** $O(n!)$ — impractical.

Approach 2 — Interval DP with Memoization

- Key insight: Think of which balloon to burst **last** in a range.
- **Time:** $O(n^3)$ — **Space:** $O(n^2)$

Approach 3 — Tabulation

- **Time:** $O(n^3)$ — **Space:** $O(n^2)$

Recommended Approach: Interval DP (think about the last balloon to burst).

Key Observation

Trick: Instead of thinking which balloon to burst first (which changes boundaries unpredictably), think which balloon to burst **last** in range $[i, j]$.

Add virtual balloons with value 1 at both ends: `nums = [1] + nums + [1]`.

`dp[i][j]` = max coins from bursting all balloons between i and j (exclusive).

```
dp[i][j] = max(
    dp[i][k] + dp[k][j] + nums[i] * nums[k] * nums[j]
) for all k in (i+1, j-1)
```

When balloon k is burst last in (i, j) , its neighbors are i and j .

Algorithm

1. Pad `nums` with 1 on both sides.

2. `dp[i][j] = 0` for all.

For each length from 2 to $n+1$: - For each left boundary i :

- `j = i + length`
- For each k between $i+1$ and $j-1$:
- `dp[i][j] = max(dp[i][j], dp[i][k] + dp[k][j] + nums[i]*nums[k]*nums[j])`

4. Return `dp[0][n+1]`.

Python Code

Recursion:

```
def maxCoins(nums):
    nums = [1] + nums + [1]
    def solve(i, j):
        if j - i < 2:
            return 0
        res = 0
        for k in range(i + 1, j):
            res = max(res, solve(i, k) + solve(k, j) + nums[i] * nums[k] * nums[j])
    return solve(0, len(nums) - 1)
```

```
        return res
    return solve(0, len(nums) - 1)
```

Memoization:

```
def maxCoins(nums):
    nums = [1] + nums + [1]
    n = len(nums)
    memo = {}

    def solve(i, j):
        if (i, j) in memo:
            return memo[(i, j)]
        if j - i < 2:
            return 0
        res = 0
        for k in range(i + 1, j):
            res = max(res, solve(i, k) + solve(k, j) + nums[i] * nums[k] * nums[j])
        memo[(i, j)] = res
        return res
    return solve(0, n - 1)
```

Tabulation:

```
def maxCoins(nums):
    nums = [1] + nums + [1]
    n = len(nums)
    dp = [[0] * n for _ in range(n)]
    for length in range(2, n):
        for i in range(n - length):
            j = i + length
            for k in range(i + 1, j):
                dp[i][j] = max(dp[i][j], dp[i][k] + dp[k][j] + nums[i] * nums[k] * nums[j])
    return dp[0][n - 1]
```

Complexity

- **Time:** $O(n^3)$
 - **Space:** $O(n^2)$
-

LC 546. Remove Boxes

Difficulty: Hard **Pattern:** Interval DP (Advanced)

Problem

Given several boxes with colors represented by integers, remove boxes to earn points. Removing k consecutive boxes of the same color earns $k * k$ points. Return the **maximum points**.

Key Observation

This is an advanced Interval DP. The state needs three dimensions: $dp[i][j][k]$ where k is the count of boxes with same color as $boxes[i]$ attached before it.

State: $dp[i][j][k]$ = max points removing $boxes[i..j]$ with k extra boxes of the same color as $boxes[i]$ already attached to the left.

Transition: 1. Remove $boxes[i]$ together with k attached boxes: $(k+1)^2 + dp[i+1][j][0]$ 2. Find some $boxes[m]$ (where $m > i$ and $boxes[m] == boxes[i]$): merge them by removing $boxes[i+1..m-1]$ first: $dp[i+1][m-1][0] + dp[m][j][k+1]$

Python Code

Memoization (Recommended for this problem):

```
def removeBoxes(boxes):
    from functools import lru_cache

    @lru_cache(None)
    def solve(i, j, k):
        if i > j:
            return 0

        # Optimization: merge consecutive same-colored boxes
        while i < j and boxes[i] == boxes[i + 1]:
```

```

        i += 1
        k += 1
    # Option 1: remove boxes[i] with k attached
    res = (k + 1) ** 2 + solve(i + 1, j, 0)
    # Option 2: find matching color further right
    for m in range(i + 2, j + 1):
        if boxes[m] == boxes[i]:
            res = max(res, solve(i + 1, m - 1, 0) + solve(m, j, k + 1))
    return res

return solve(0, len(boxes) - 1, 0)

```

Complexity

- **Time:** $O(n^3)$
- **Space:** $O(n^3)$

Pattern 36: DP - Catalan Numbers

LC 96. Unique Binary Search Trees

Difficulty: Medium **Pattern:** Catalan Numbers

Problem

Given n , return the number of **structurally unique BSTs** that store values 1 to n .

Approaches

Approach 1 — Recursion

- For each root i (1 to n): left subtree has $i-1$ nodes, right has $n-i$ nodes.
- $G(n) = \sum G(i-1) \times G(n-i)$ for $i = 1$ to n .
- This is the **Catalan number** formula.
- **Time:** $O(3^n)$ — **Space:** $O(n)$

Approach 2 — Memoization

- **Time:** $O(n^2)$ — **Space:** $O(n)$

Approach 3 — Tabulation

- **Time:** $O(n^2)$ — **Space:** $O(n)$

Recommended Approach: Tabulation. Also know the mathematical formula.

Key Observation

This is the n th Catalan number:

$$C(n) = \sum C(i) \times C(n-1-i) \text{ for } i = 0 \text{ to } n-1$$
$$C(0) = C(1) = 1$$

Mathematical formula: $C(n) = C(2n, n) / (n+1) = (2n)! / ((n+1)! \times n!)$

Python Code

Recursion:

```
def numTrees(n):
    if n <= 1:
        return 1
    total = 0
    for i in range(n):
        total += numTrees(i) * numTrees(n - 1 - i)
    return total
```

Memoization:

```
def numTrees(n):
    memo = {}
    def solve(n):
        if n in memo:
            return memo[n]
        if n <= 1:
```

```

        return 1
    total = 0
    for i in range(n):
        total += solve(i) * solve(n - 1 - i)
    memo[n] = total
    return total
return solve(n)

```

Tabulation:

```

def numTrees(n):
    dp = [0] * (n + 1)
    dp[0] = dp[1] = 1
    for nodes in range(2, n + 1):
        for i in range(nodes):
            dp[nodes] += dp[i] * dp[nodes - 1 - i]
    return dp[n]

```

Dry Run

n	dp[n]	Computation
0	1	base
1	1	base
2	2	$dp[0] \times dp[1] + dp[1] \times dp[0]$ $= 1+1$
3	5	$dp[0] \times dp[2] + dp[1] \times dp[1]$ $+ dp[2] \times dp[0] = 2+1+2$
4	14	$5+2+2+5$

Complexity

- **Time:** $O(n^2)$
 - **Space:** $O(n)$
-

LC 95. Unique Binary Search Trees II

Difficulty: Medium **Pattern:** Catalan Numbers (Construct All)

Problem

Given n , return **all structurally unique BSTs** that store values 1 to n . Return the root nodes.

Key Observation

Same structure as LC 96, but instead of counting, **construct** all trees. For each root i , recursively generate all left subtrees from $[start, i-1]$ and right subtrees from $[i+1, end]$, then combine.

Python Code

Recursion (inherently constructs all — no need for tabulation):

```
def generateTrees(n):
    def solve(start, end):
        if start > end:
            return [None]
        trees = []
        for i in range(start, end + 1):
            left_trees = solve(start, i - 1)
            right_trees = solve(i + 1, end)
            for l in left_trees:
                for r in right_trees:
                    root = TreeNode(i)
                    root.left = l
                    root.right = r
                    trees.append(root)
```

```
    return trees
return solve(1, n)
```

Memoization:

```
def generateTrees(n):
    memo = {}
    def solve(start, end):
        if (start, end) in memo:
            return memo[(start, end)]
        if start > end:
            return [None]
        trees = []
        for i in range(start, end + 1):
            for l in solve(start, i - 1):
                for r in solve(i + 1, end):
                    root = TreeNode(i)
                    root.left = l
                    root.right = r
                    trees.append(root)
        memo[(start, end)] = trees
    return trees
return solve(1, n)
```

Complexity

- **Time:** $O(n \times C(n))$ where $C(n)$ is the n th Catalan number ($\sim 4^n / n^{1.5}$)
- **Space:** $O(n \times C(n))$ for storing all trees

LC 241. Different Ways to Add Parentheses

Difficulty: Medium **Pattern:** Catalan / Interval DP

Problem

Given a string expression of numbers and operators (+, -, *), return **all possible results** from computing all the different ways to group sub-expressions.

Key Observation

For each operator, split the expression into left and right parts. Recursively compute all results for each part, then combine. This is a divide-and-conquer / interval DP pattern closely related to Catalan structure.

Python Code

Recursion:

```
def diffWaysToCompute(expression):
    def solve(expr):
        results = []
        for i, ch in enumerate(expr):
            if ch in '+-*':
                left = solve(expr[:i])
                right = solve(expr[i+1:])
                for l in left:
                    for r in right:
                        if ch == '+':
                            results.append(l + r)
                        elif ch == '-':
                            results.append(l - r)
                        else:
                            results.append(l * r)
                if not results:
                    results.append(int(expr))
        return results
    return solve(expression)
```

Memoization:

```
def diffWaysToCompute(expression):
    memo = {}
    def solve(expr):
        if expr in memo:
            return memo[expr]
        results = []
```

```

    for i, ch in enumerate(expr):
        if ch in '+-*':
            left = solve(expr[:i])
            right = solve(expr[i+1:])
            for l in left:
                for r in right:
                    if ch == '+':
                        results.append(l + r)
                    elif ch == '-':
                        results.append(l - r)
                    else:
                        results.append(l * r)
            if not results:
                results.append(int(expr))
            memo[expr] = results
            return results
    return solve(expression)

```

Complexity

- **Time:** $O(C(n))$ — Catalan number of ways to parenthesize
- **Space:** $O(C(n))$ for storing results

Pattern 37: DP - Longest Increasing Subsequence (LIS)

LC 300. Longest Increasing Subsequence

Difficulty: Medium **Pattern:** LIS

Problem

Given an integer array `nums`, return the length of the **longest strictly increasing subsequence**.

Approaches

Very important. Know all approaches including the $O(n \log n)$ one.

Approach 1 — Recursion

- For each element, include (if valid) or exclude.
- **Time:** $O(2^n)$ — **Space:** $O(n)$

Approach 2 — Memoization

- State: `(index, prev_index)`.
- **Time:** $O(n^2)$ — **Space:** $O(n^2)$

Approach 3 — Tabulation $O(n^2)$

- `dp[i]` = length of LIS ending at index `i`.
- `dp[i] = max(dp[j] + 1)` for all `j < i` where `nums[j] < nums[i]`.
- **Time:** $O(n^2)$ — **Space:** $O(n)$

Approach 4 — Binary Search $O(n \log n)$ (Patience Sorting)

- Maintain a `tails` array. For each number, use `bisect_left` to find position.
- **Time:** $O(n \log n)$ — **Space:** $O(n)$

Recommended Approach: $O(n \log n)$ for optimal, but know $O(n^2)$ for interviews.

Key Observation ($O(n \log n)$)

Maintain array `tails` where `tails[i]` = smallest tail element of all increasing subsequences of length `i+1`.

For each number: - Binary search for its position in `tails`. - If position == `len(tails)` → append (extends longest). - Else → replace `tails[pos]` (creates potential for longer subsequence).

`tails` is always sorted, so binary search works.

Python Code

Recursion:

```

def lengthOfLIS(nums):
    def solve(i, prev):
        if i == len(nums):
            return 0
        skip = solve(i + 1, prev)
        take = 0
        if prev == -1 or nums[i] > nums[prev]:
            take = 1 + solve(i + 1, i)
        return max(skip, take)
    return solve(0, -1)

```

Memoization:

```

def lengthOfLIS(nums):
    memo = {}
    def solve(i, prev):
        if (i, prev) in memo:
            return memo[(i, prev)]
        if i == len(nums):
            return 0
        skip = solve(i + 1, prev)
        take = 0
        if prev == -1 or nums[i] > nums[prev]:
            take = 1 + solve(i + 1, i)
        memo[(i, prev)] = max(skip, take)
        return memo[(i, prev)]
    return solve(0, -1)

```

Tabulation $O(n^2)$:

```

def lengthOfLIS(nums):
    n = len(nums)
    dp = [1] * n
    for i in range(1, n):
        for j in range(i):
            if nums[j] < nums[i]:

```

```
        dp[i] = max(dp[i], dp[j] + 1)
    return max(dp)
```

Binary Search $O(n \log n)$:

```
from bisect import bisect_left

def lengthOfLIS(nums):
    tails = []
    for num in nums:
        pos = bisect_left(tails, num)
        if pos == len(tails):
            tails.append(num)
        else:
            tails[pos] = num
    return len(tails)
```

Dry Run (Binary Search)

Input: `nums = [10, 9, 2, 5, 3, 7, 101, 18]`

num	tails	Action
10	[10]	append
9	[9]	replace 10
2	[2]	replace 9
5	[2, 5]	append
3	[2, 3]	replace 5
7	[2, 3, 7]	append

101	[2, 3, 7, 101]	append
18	[2, 3, 7, 18]	replace 101

Answer: 4 (e.g., [2, 3, 7, 101])

Complexity

- **Time:** $O(n \log n)$ — binary search approach
- **Space:** $O(n)$

LC 354. Russian Doll Envelopes

Difficulty: Hard **Pattern:** LIS (2D)

Problem

Given `envelopes[i] = [width, height]`, find the **maximum number** of envelopes you can nest (Russian doll style). An envelope fits inside another if both width and height are strictly greater.

Key Observation

1. **Sort** by width ascending. If widths are equal, sort by height **descending**.
2. Run **LIS on heights only**.

Why sort height descending when widths equal? Because we can't use two envelopes with the same width. Descending order ensures LIS won't pick two same-width envelopes.

Python Code

Recursion-based LIS (after sorting):

```
def maxEnvelopes(envelopes):
    envelopes.sort(key=lambda x: (x[0], -x[1]))
    heights = [e[1] for e in envelopes]
```

```

def solve(i, prev):
    if i == len(heights):
        return 0
    skip = solve(i + 1, prev)
    take = 0
    if prev == -1 or heights[i] > heights[prev]:
        take = 1 + solve(i + 1, i)
    return max(skip, take)
return solve(0, -1)

```

Optimal (Sort + LIS with Binary Search):

```

from bisect import bisect_left

def maxEnvelopes(envelopes):
    envelopes.sort(key=lambda x: (x[0], -x[1]))
    tails = []
    for _, h in envelopes:
        pos = bisect_left(tails, h)
        if pos == len(tails):
            tails.append(h)
        else:
            tails[pos] = h
    return len(tails)

```

Complexity

- **Time:** $O(n \log n)$
- **Space:** $O(n)$

LC 1671. Minimum Number of Removals to Make Mountain Array

Difficulty: Hard **Pattern:** LIS (Bidirectional)

Problem

Given an array `nums`, return the **minimum number of elements to remove** to make it a **mountain array**. A mountain array has elements that strictly increase then strictly decrease, with at least 3 elements.

Key Observation

A mountain at peak index `i` has: - LIS from left ending at `i` (call it `lis[i]`) - LDS from right ending at `i` (= LIS from right, call it `lds[i]`) - Mountain length at `i` = `lis[i] + lds[i] - 1` (subtract 1 for counting peak twice) - Both `lis[i]` and `lds[i]` must be ≥ 2 .

Answer = `n - max(mountain length at each valid i)`.

Python Code

Recursion (computing LIS from both directions):

```
def minimumMountainRemovals(nums):
    n = len(nums)

    # LIS ending at each index (left to right) using O(n^2)
    lis = [1] * n
    for i in range(1, n):
        for j in range(i):
            if nums[j] < nums[i]:
                lis[i] = max(lis[i], lis[j] + 1)

    # LIS ending at each index (right to left) = LDS
    lds = [1] * n
    for i in range(n - 2, -1, -1):
        for j in range(n - 1, i, -1):
            if nums[j] < nums[i]:
                lds[i] = max(lds[i], lds[j] + 1)

    max_mountain = 0
    for i in range(1, n - 1):
        if lis[i] >= 2 and lds[i] >= 2:
            max_mountain = max(max_mountain, lis[i] + lds[i] - 1)
    return n - max_mountain
```

Optimal (Binary Search for LIS in both directions):

```
from bisect import bisect_left

def minimumMountainRemovals(nums):
    n = len(nums)

    def get_lis_lengths(arr):
        tails = []
        lengths = [0] * len(arr)
        for i, num in enumerate(arr):
            pos = bisect_left(tails, num)
            if pos == len(tails):
                tails.append(num)
            else:
                tails[pos] = num
            lengths[i] = pos + 1
        return lengths

    lis = get_lis_lengths(nums)
    lds = get_lis_lengths(nums[::-1])[::-1]

    max_mountain = 0
    for i in range(1, n - 1):
        if lis[i] >= 2 and lds[i] >= 2:
            max_mountain = max(max_mountain, lis[i] + lds[i] - 1)
    return n - max_mountain
```

Complexity

- **Time:** $O(n \log n)$
- **Space:** $O(n)$

LC 2407. Longest Increasing Subsequence II

Difficulty: Hard **Pattern:** LIS with Constraint + Segment Tree

Problem

Given `nums` and integer `k`, find the length of the **longest strictly increasing subsequence** such that the difference between adjacent elements in the subsequence is **at most k**.

Key Observation

Standard LIS won't work because of the constraint on difference. For each `nums[i]`, we need $\max(dp[j])$ for all `j` where `nums[i] - k ≤ nums[j] < nums[i]`. This is a **range maximum query** → use a **Segment Tree**.

Approach

1. `dp[v]` = length of longest valid subsequence ending with value `v`.
2. For each `nums[i]`, query segment tree for max in range `[nums[i] - k, nums[i] - 1]`.
3. Update `dp[nums[i]] = query_result + 1`.
4. Answer = max of all `dp` values.

Python Code

Segment Tree Solution:

```
def lengthOfLIS(nums, k):
    max_val = max(nums)
    tree = [0] * (4 * (max_val + 1))

    def update(node, start, end, idx, val):
        if start == end:
            tree[node] = val
            return
        mid = (start + end) // 2
        if idx <= mid:
            update(2 * node, start, mid, idx, val)
        else:
            update(2 * node + 1, mid + 1, end, idx, val)
        tree[node] = max(tree[2 * node], tree[2 * node + 1])
```

```

def query(node, start, end, l, r):
    if r < start or end < l:
        return 0
    if l <= start and end <= r:
        return tree[node]
    mid = (start + end) // 2
    return max(query(2 * node, start, mid, l, r),
               query(2 * node + 1, mid + 1, end, l, r))

ans = 0
for num in nums:
    lo = max(1, num - k)
    hi = num - 1
    best = query(1, 1, max_val, lo, hi) if lo <= hi else 0
    best += 1
    ans = max(ans, best)
    update(1, 1, max_val, num, best)
return ans

```

Complexity

- **Time:** $O(n \log M)$ where M = max value in nums
- **Space:** $O(M)$

PART C — CROSS-PROBLEM PATTERN CONNECTIONS

Fibonacci Style Connection

Fibonacci Number

→ Base pattern: $dp[i] = dp[i-1] + dp[i-2]$

Climbing Stairs

→ Same as Fibonacci with $dp[1]=1$, $dp[2]=2$

Min Cost Climbing Stairs

→ Fibonacci + min cost selection

Decode Ways

→ Fibonacci variant with validity checks

House Robber

→ Fibonacci with max(skip, take)

House Robber II

→ Circular → two passes of House Robber

House Robber III

→ Tree version → DFS returning (rob, skip) pair

Delete and Earn

→ Transform to House Robber on value frequency array

Knapsack Family Connection

Coin Change (min coins)

→ Unbounded knapsack, minimize

Coin Change II (count combinations)

→ Unbounded knapsack, outer=coins inner=amounts

Combination Sum IV (count permutations)

→ Unbounded knapsack, outer=amounts inner=nums

Partition Equal Subset Sum

→ 0/1 knapsack, boolean (can we reach target?)

Target Sum

→ 0/1 knapsack, count (how many subsets reach target?)

→ Mathematical reduction: $P = (\text{target} + \text{total}) / 2$

String DP Connection

Word Break

→ Can string be segmented? Boolean DP

Word Break II

→ Return all segmentations → DP + backtracking

LCS

→ Match characters or skip

Delete Operation for Two Strings

→ LCS reduction: $\text{deletions} = m + n - 2 * \text{LCS}$

Edit Distance

→ LCS extension with insert/delete/replace

Grid DP Connection

Unique Paths

→ Count paths: $\text{dp}[i][j] = \text{up} + \text{left}$

Unique Paths II

→ Same + obstacle handling

Minimum Path Sum

→ Min cost: $\text{dp}[i][j] = \text{grid}[i][j] + \min(\text{up}, \text{left})$

Triangle

→ Non-rectangular grid, process bottom-up

Minimum Falling Path Sum

→ Column movement includes diagonals

Maximal Square

→ $\text{dp}[i][j] = \min(\text{up}, \text{left}, \text{diagonal}) + 1$

Count Square Submatrices

→ Same as Maximal Square, sum all dp values

LIS Connection

LIS

→ Base pattern: $O(n^2)$ DP or $O(n \log n)$ binary search

Russian Doll Envelopes

→ Sort by width, LIS on heights (descending height for same width)

Mountain Array Removals

→ LIS from left + LIS from right, find best peak

LIS II (with constraint k)

→ Range max query → Segment Tree

Interval DP / Catalan Connection

Unique BSTs (count)

→ Catalan number: $dp[n] = \sum dp[i] * dp[n-1-i]$

Unique BSTs II (construct)

→ Same structure, build all trees

Different Ways to Add Parentheses

→ Split at each operator, Catalan structure

Burst Balloons

→ Interval DP: which balloon to burst LAST

Remove Boxes

→ Advanced interval DP: 3D state with attached count

PART D — FINAL REVISION SHEET

Pattern Recognition Cheat Sheet

If the problem says...	Think...
<code>dp[i]</code> depends on few previous values	Fibonacci Style DP
Maximum/minimum contiguous subarray	Kadane's Algorithm
Unlimited items, make target sum	Unbounded Knapsack / Coin Change
Count combinations (order doesn't matter)	Outer: items, Inner: amounts
Count permutations (order matters)	Outer: amounts, Inner: items
Each item at most once, subset	0/1 Knapsack (iterate right→left)
Partition into two equal subsets	Subset Sum → total/2
Assign +/- to reach target	Target Sum → subset sum reduction
Segment string into dictionary words	Word Break
Two strings, common subsequence	LCS
Transform one string to another	Edit Distance
Grid paths, min cost	Grid DP

Largest square of 1s	Maximal Square: $\min(\text{up}, \text{left}, \text{diag}) + 1$
Optimal merge/split range	Interval DP
Count unique structures/trees	Catalan Numbers
Longest subsequence with ordering	LIS
Adjacent difference constraint in LIS	Segment Tree + LIS
Circular arrangement (can't use first+last)	Two passes excluding each end
Tree DP (can't use parent+child)	DFS returning (take, skip) pair

DP Approach Decision Flowchart

1. Can I define the state clearly?

→ Yes → Continue

→ No → Rethink the problem

2. Does the state have overlapping subproblems?

→ Yes → DP

→ No → Divide & Conquer or Greedy

3. Start with Recursion → Add Memo → Convert to Tabulation → Optimize Space

4. For 0/1 problems: iterate RIGHT to LEFT in 1D

5. For unbounded problems: iterate LEFT to RIGHT in 1D

6. For combinations: outer loop = items

7. For permutations: outer loop = target values

Quick Complexity Reference

Problem	Time	Space
Fibonacci / Climbing Stairs	$O(n)$	$O(1)$
House Robber	$O(n)$	$O(1)$
Decode Ways	$O(n)$	$O(1)$
Maximum Subarray	$O(n)$	$O(1)$
Coin Change	$O(n \times \text{amount})$	$O(\text{amount})$
Coin Change II	$O(n \times \text{amount})$	$O(\text{amount})$
Partition Equal Subset	$O(n \times \text{sum}/2)$	$O(\text{sum}/2)$
Target Sum	$O(n \times \text{target})$	$O(\text{target})$
Word Break	$O(n^2)$	$O(n)$
LCS	$O(m \times n)$	$O(n)$
Edit Distance	$O(m \times n)$	$O(n)$
Unique Paths	$O(m \times n)$	$O(n)$
Maximal Square	$O(m \times n)$	$O(n)$
Burst Balloons	$O(n^3)$	$O(n^2)$
Unique BSTs	$O(n^2)$	$O(n)$

LIS	$O(n \log n)$	$O(n)$
Russian Doll Envelopes	$O(n \log n)$	$O(n)$